PROGRAMACION 3

Iniciar grabación

Grafos p2

Programación 3 TUDAI

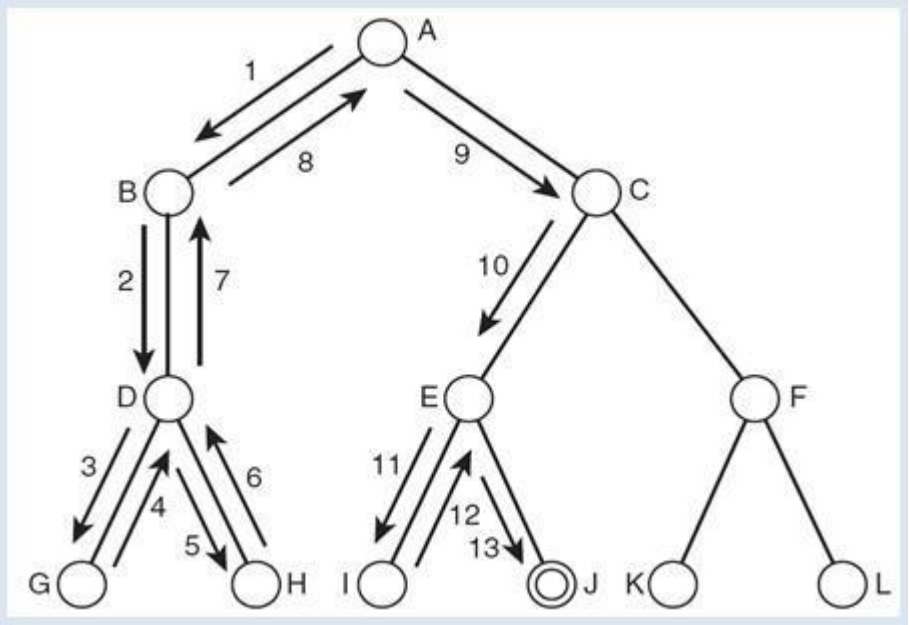
Recorridos de un Grafo

Un recorrido (o exploración) de un Grafo visita todos los vértices en una secuencia determinada.

Vamos a ver dos de los recorridos más comunes, el recorrido en profundidad (DFS) y el recorrido en amplitud (BFS).

DFS (Depth-First Search)
Recorrido en profundidad

- Es una generalización del recorrido preorden de un árbol.



- Recursivamente visita todos los adyacentes no visitados de un vértice.

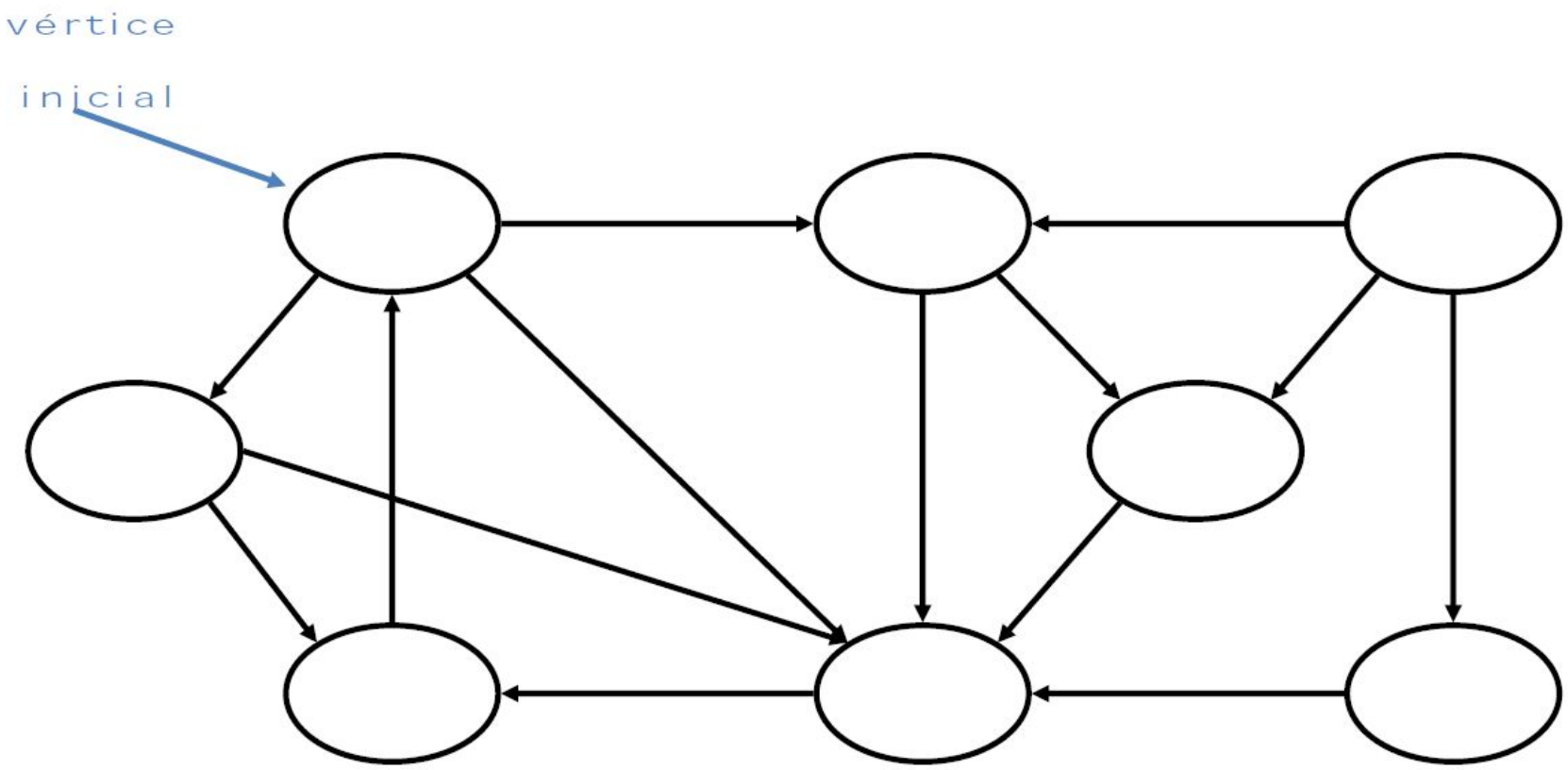
DFS: Review del Código

```
DFS(G)
{
    for each vértice u ∈ G->V
    {
        u->color = BLANCO;
    }
    tiempo= 0;
    for each vértice u ∈ G->V
    {
      if (u->color == BLANCO)
          DFS_Visit(u);
    }
}
```

```
DFS_Visit(u)
{
    u->color = AMARILLO;
    tiempo = tiempo+1;
    u->d = tiempo;
    for each v ∈ u->Adj[]
    {
        if (v->color== BLANCO)
            DFS_Visit(v);
    }
    u->color = NEGRO;
    tiempo = tiempo +1;
    u->f = tiempo;
}
```

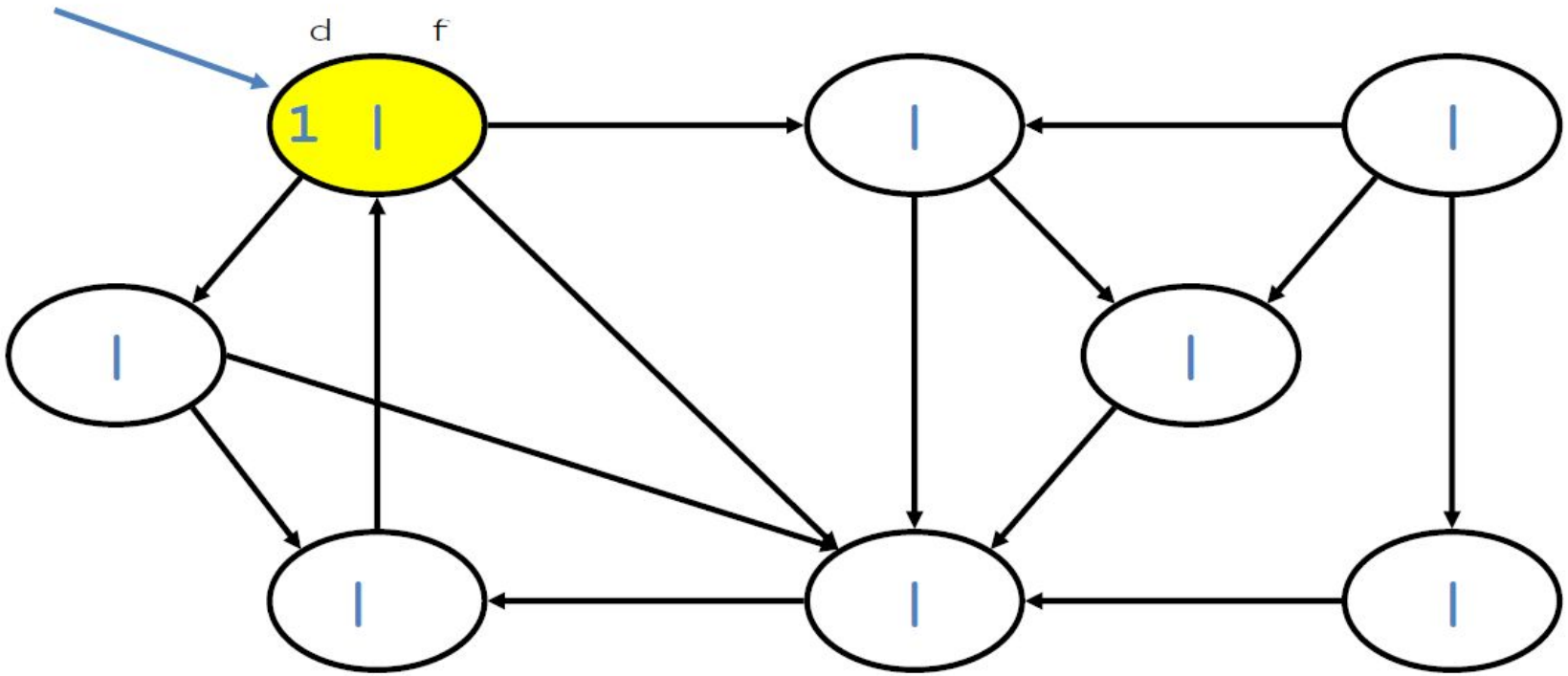
Es un planteo recursivo.

Traza de Recorrido en Profundidad DFS (1/17)



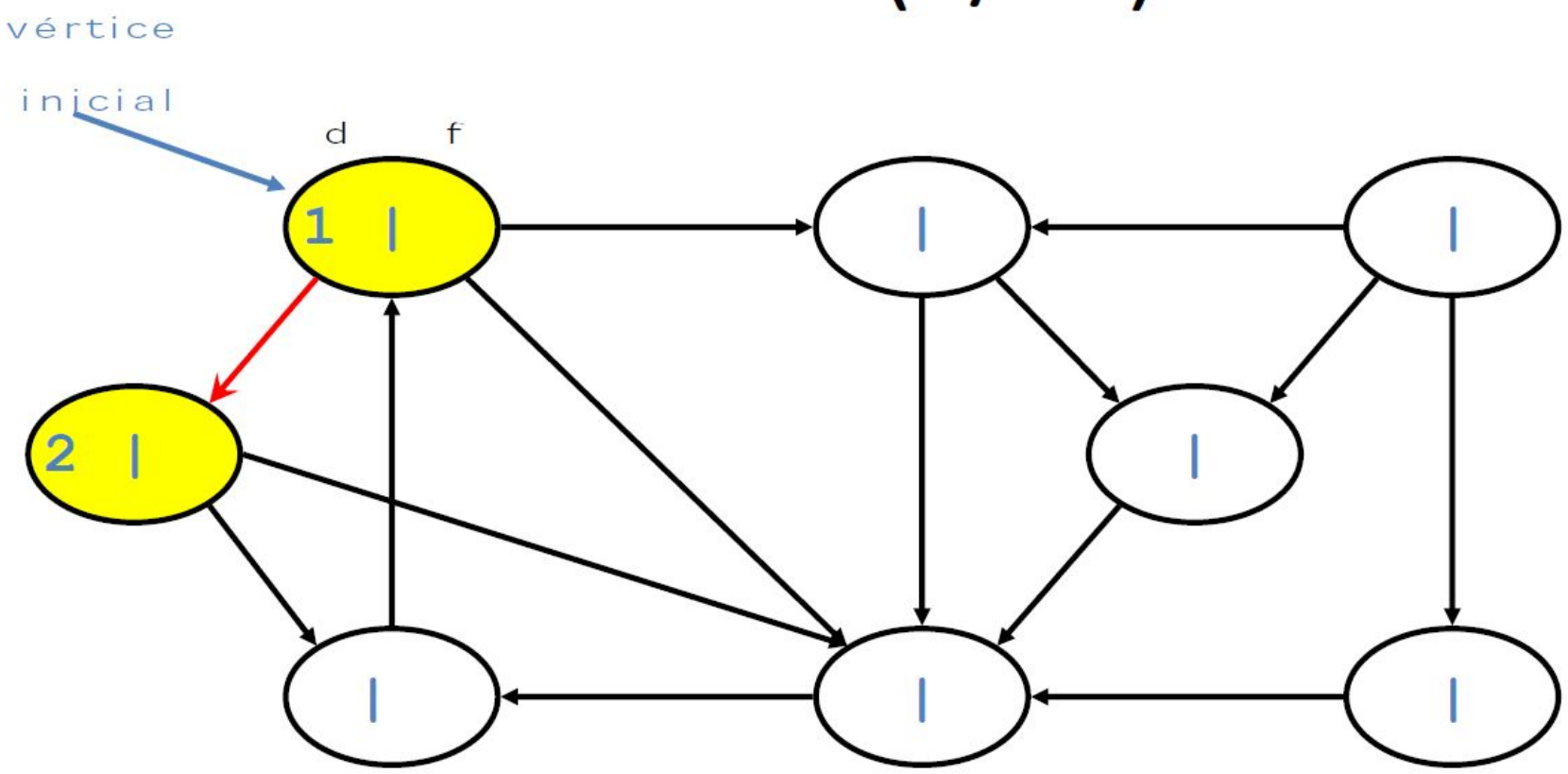

```
DFS_Visit(u)
{
   u->color = AMARILLO;
   tiempo = tiempo+1;
   u->d = tiempo;
   for each v ∈ u->Adj[]
   {
      if (v->color== BLANCO)
         DFS_Visit(v);
   }
   u->color = NEGRO;
   tiempo = tiempo +1;
   u->f = tiempo;
}
```

Traza de Recorrido en Profundidad DFS (2/17)



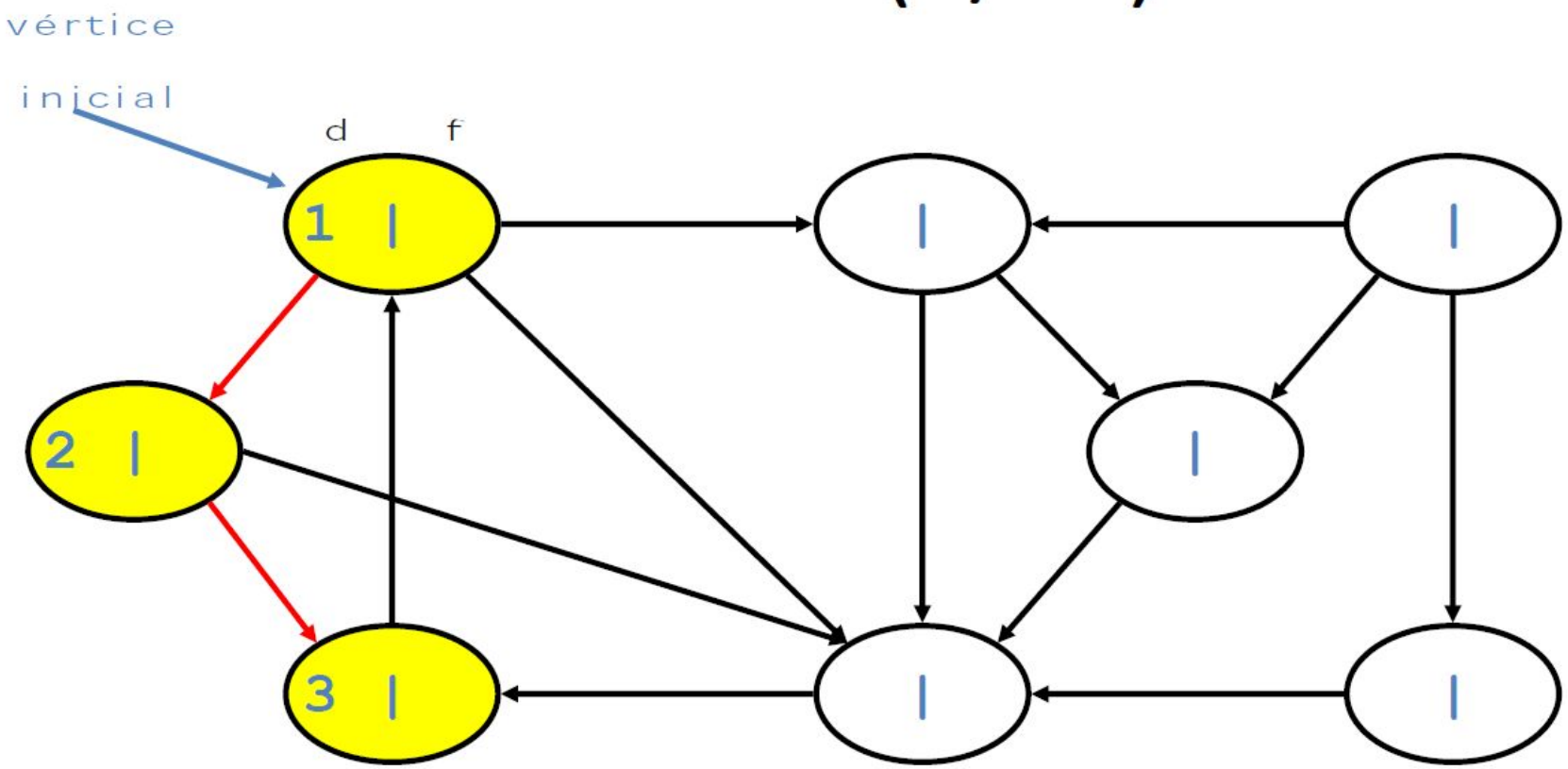

```
DFS_Visit(u)
{
    u->color = AMARILLO;
    tiempo = tiempo+1;
    u->d = tiempo;
    for each v ∈ u->Adj[]
    {
        if (v->color== BLANCO)
            DFS_Visit(v);
    }
    u->color = NEGRO;
    tiempo = tiempo +1;
    u->f = tiempo;
}
```

Traza de Recorrido en Profundidad DFS (3/17)



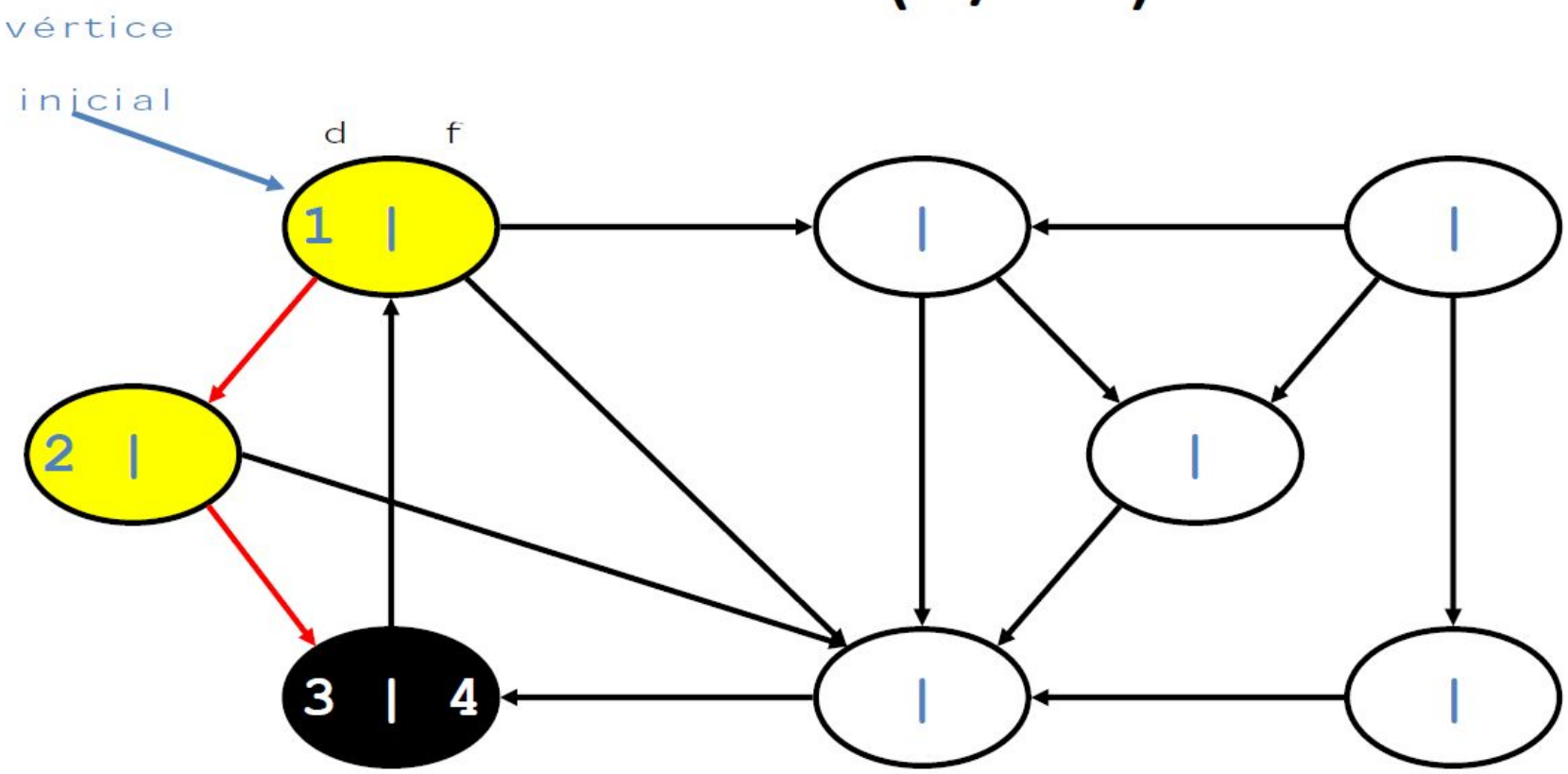

```
DFS_Visit(u)
{
    u->color = AMARILLO;
    tiempo = tiempo+1;
    u->d = tiempo;
    for each v ∈ u->Adj[]
    {
        if (v->color== BLANCO)
            DFS_Visit(v);
    }
    u->color = NEGRO;
    tiempo = tiempo +1;
    u->f = tiempo;
}
```

Traza de Recorrido en Profundidad DFS (4/17)



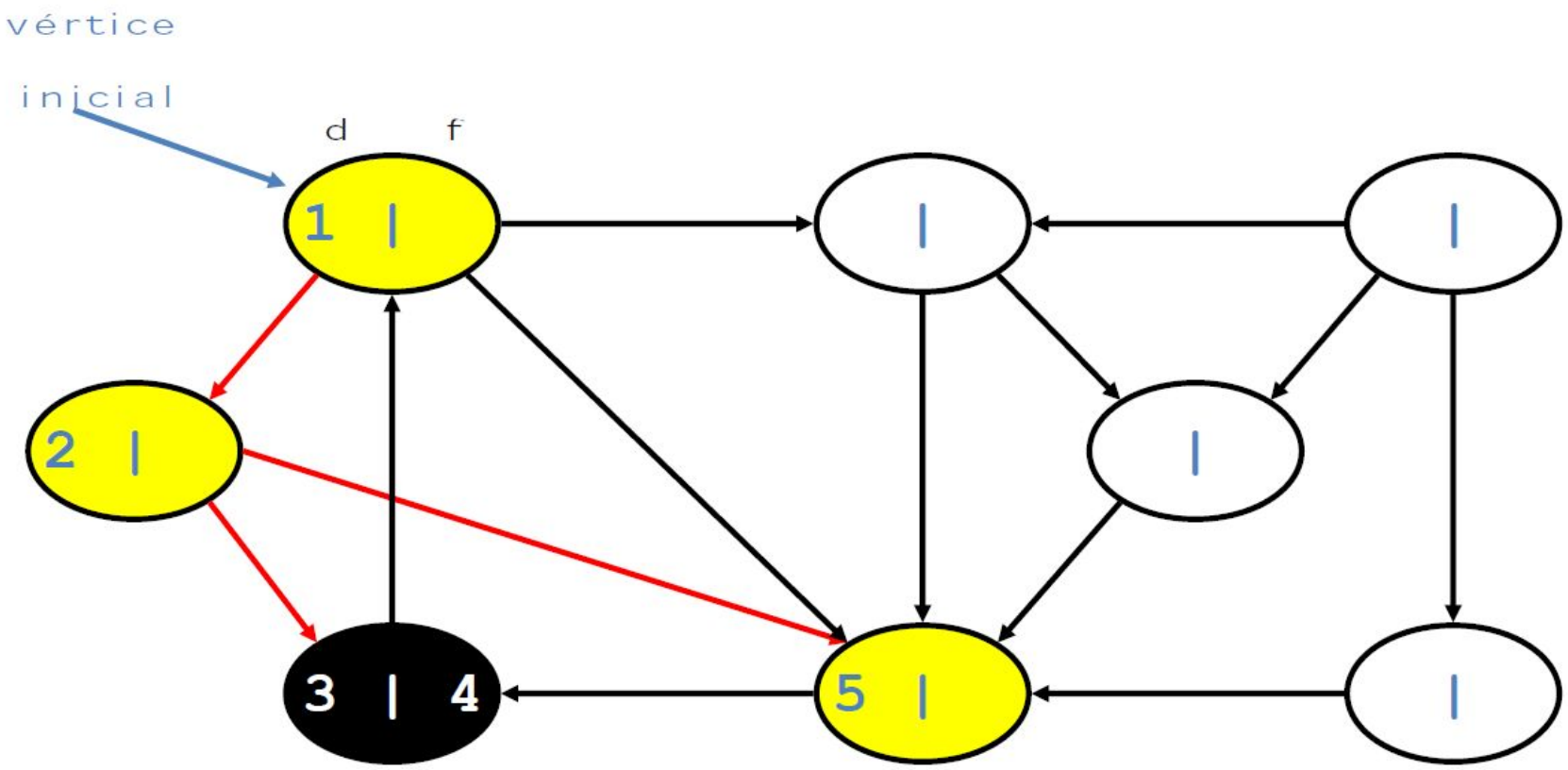

```
DFS_Visit(u)
{
    u->color = AMARILLO;
    tiempo = tiempo+1;
    u->d = tiempo;
    for each v ∈ u->Adj[]
    {
        if (v->color== BLANCO)
            DFS_Visit(v);
    }
    u->color = NEGRO;
    tiempo = tiempo +1;
    u->f = tiempo;
}
```

Traza de Recorrido en Profundidad DFS (5/17)



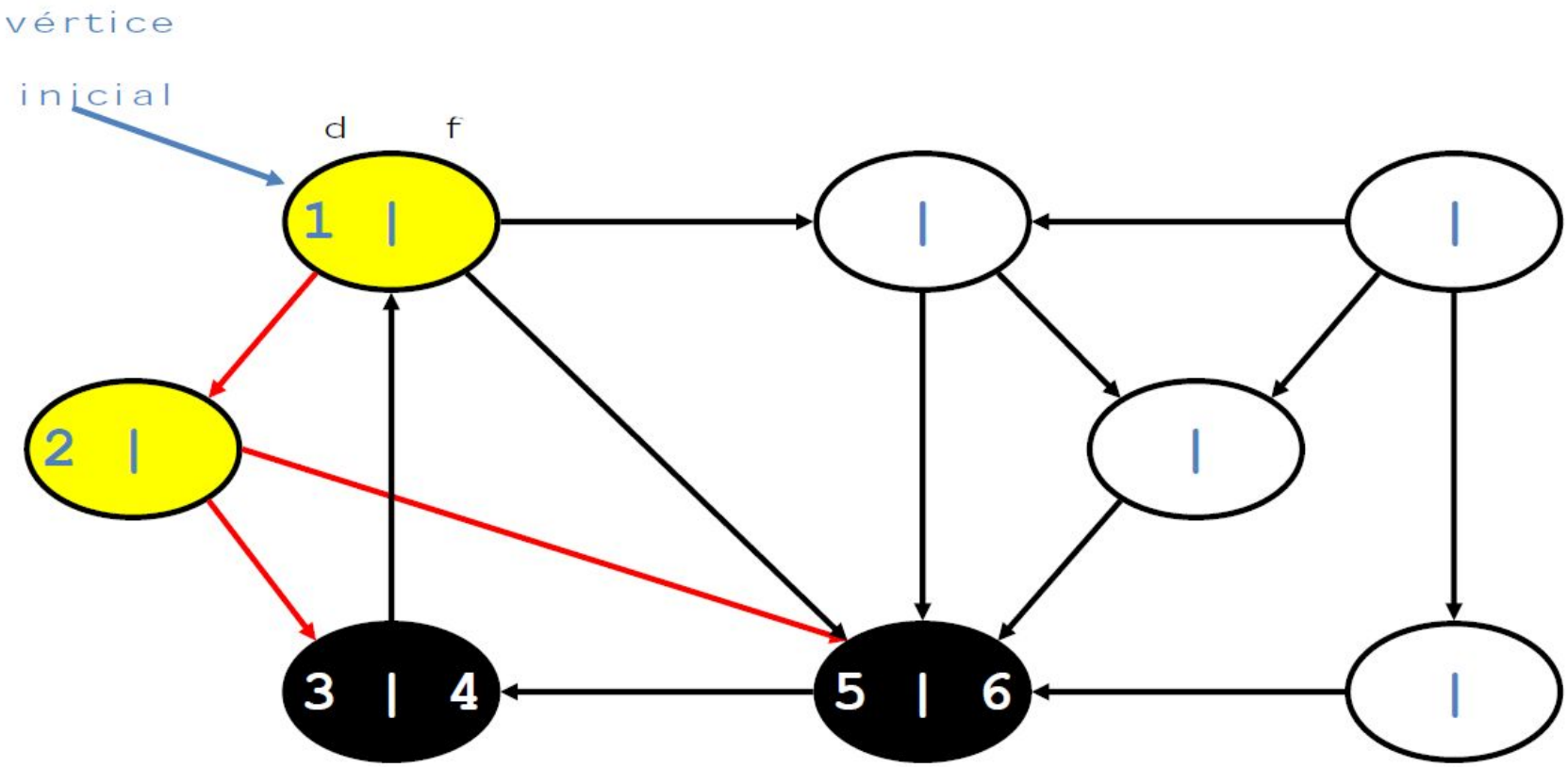

```
DFS_Visit(u)
{
    u->color = AMARILLO;
    tiempo = tiempo+1;
    u->d = tiempo;
    for each v ∈ u->Adj[]
    {
        if (v->color== BLANCO)
            DFS_Visit(v);
    }
    u->color = NEGRO;
    tiempo = tiempo +1;
    u->f = tiempo;
}
```

Traza de Recorrido en Profundidad DFS (6/17)



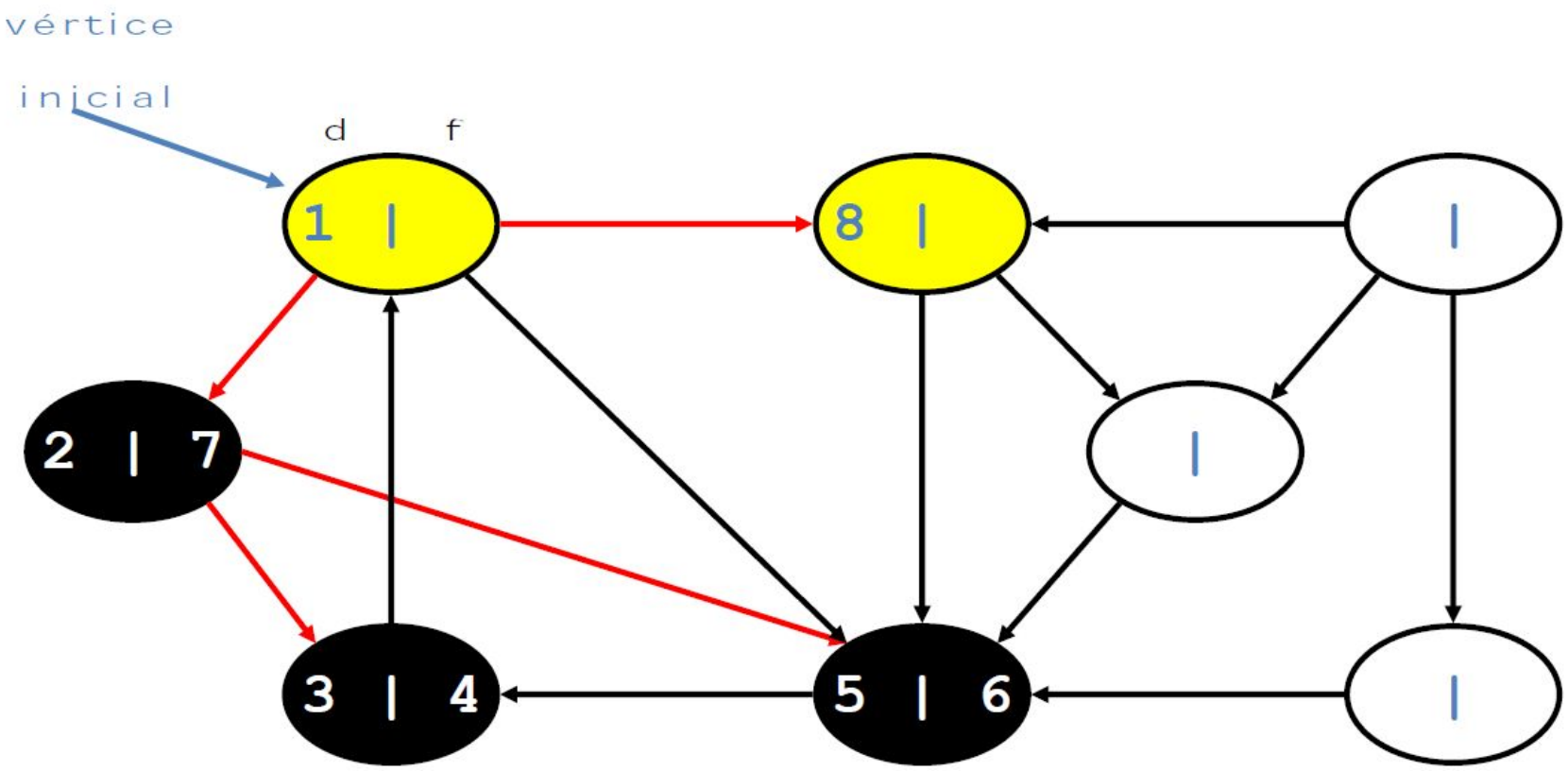

```
DFS_Visit(u)
{
    u->color = AMARILLO;
    tiempo = tiempo+1;
    u->d = tiempo;
    for each v ∈ u->Adj[]
    {
        if (v->color== BLANCO)
            DFS_Visit(v);
    }
    u->color = NEGRO;
    tiempo = tiempo +1;
    u->f = tiempo;
}
```

Traza de Recorrido en Profundidad DFS (7/17)



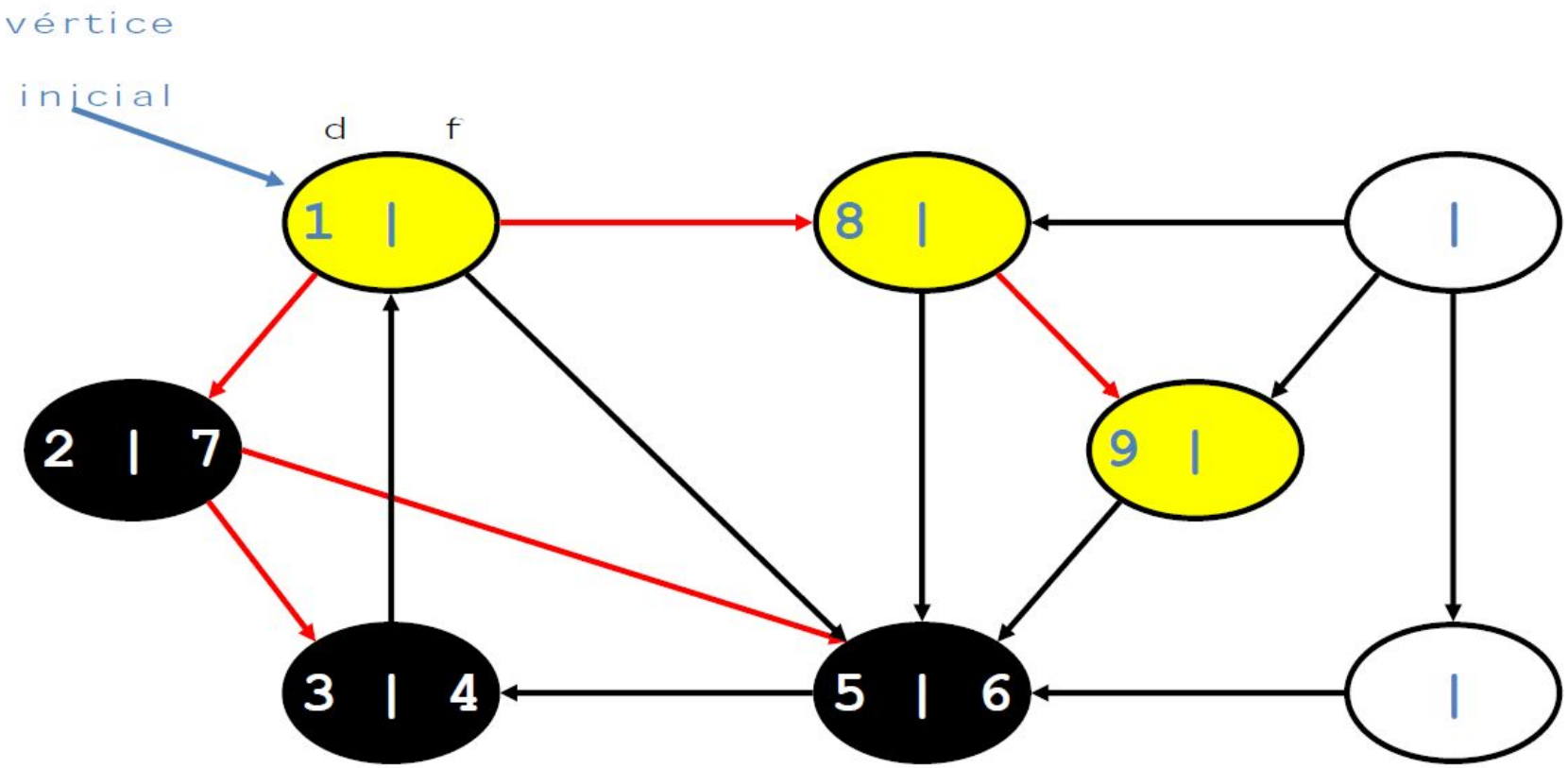

```
DFS_Visit(u)
{
    u->color = AMARILLO;
    tiempo = tiempo+1;
    u->d = tiempo;
    for each v ∈ u->Adj[]
    {
        if (v->color== BLANCO)
            DFS_Visit(v);
    }
    u->color = NEGRO;
    tiempo = tiempo +1;
    u->f = tiempo;
}
```

Traza de Recorrido en Profundidad DFS (8/17)



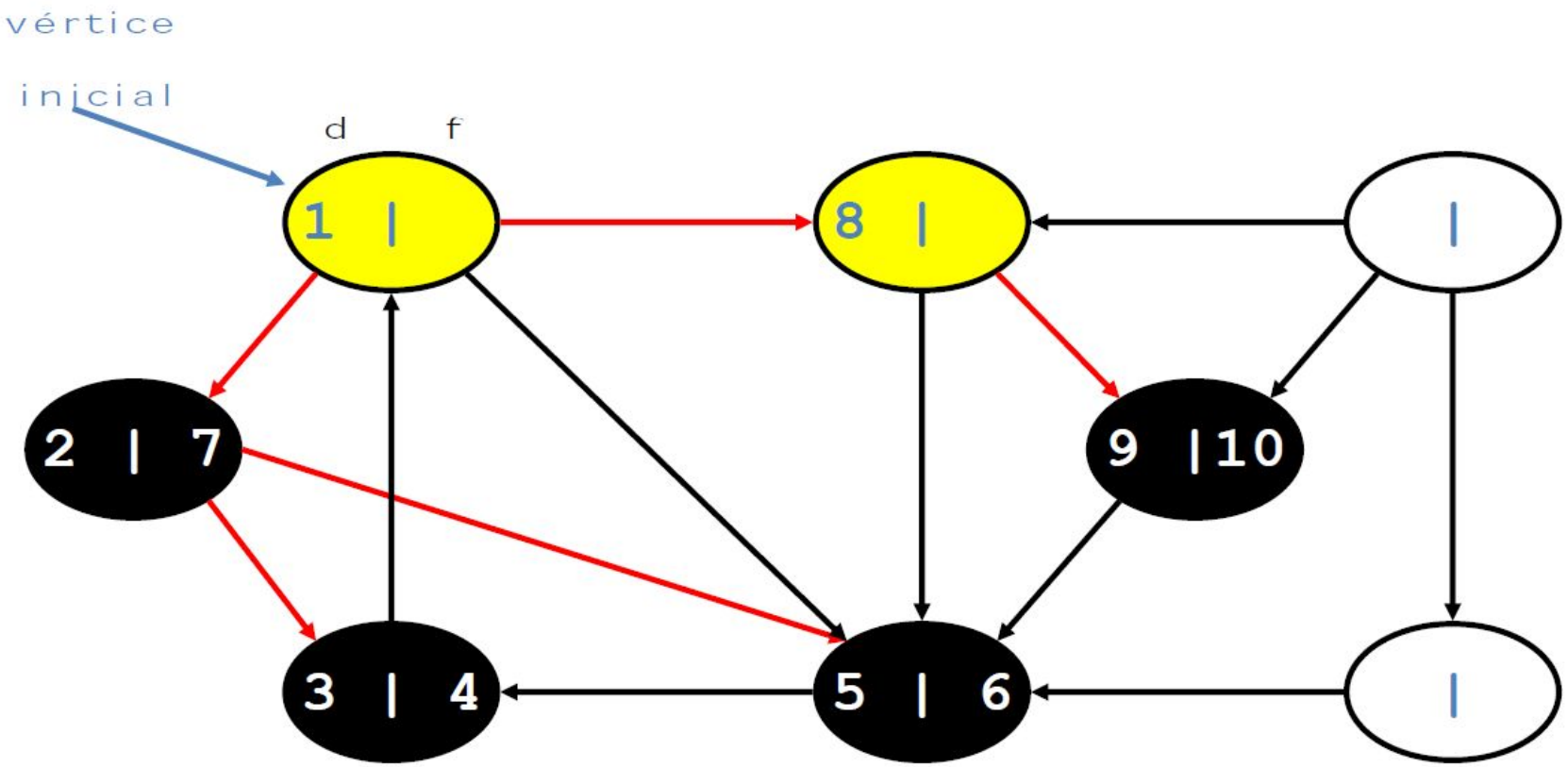

```
DFS_Visit(u)
{
    u->color = AMARILLO;
    tiempo = tiempo+1;
    u->d = tiempo;
    for each v ∈ u->Adj[]
    {
        if (v->color== BLANCO)
            DFS_Visit(v);
    }
    u->color = NEGRO;
    tiempo = tiempo +1;
    u->f = tiempo;
}
```

Traza de Recorrido en Profundidad DFS (10/17)



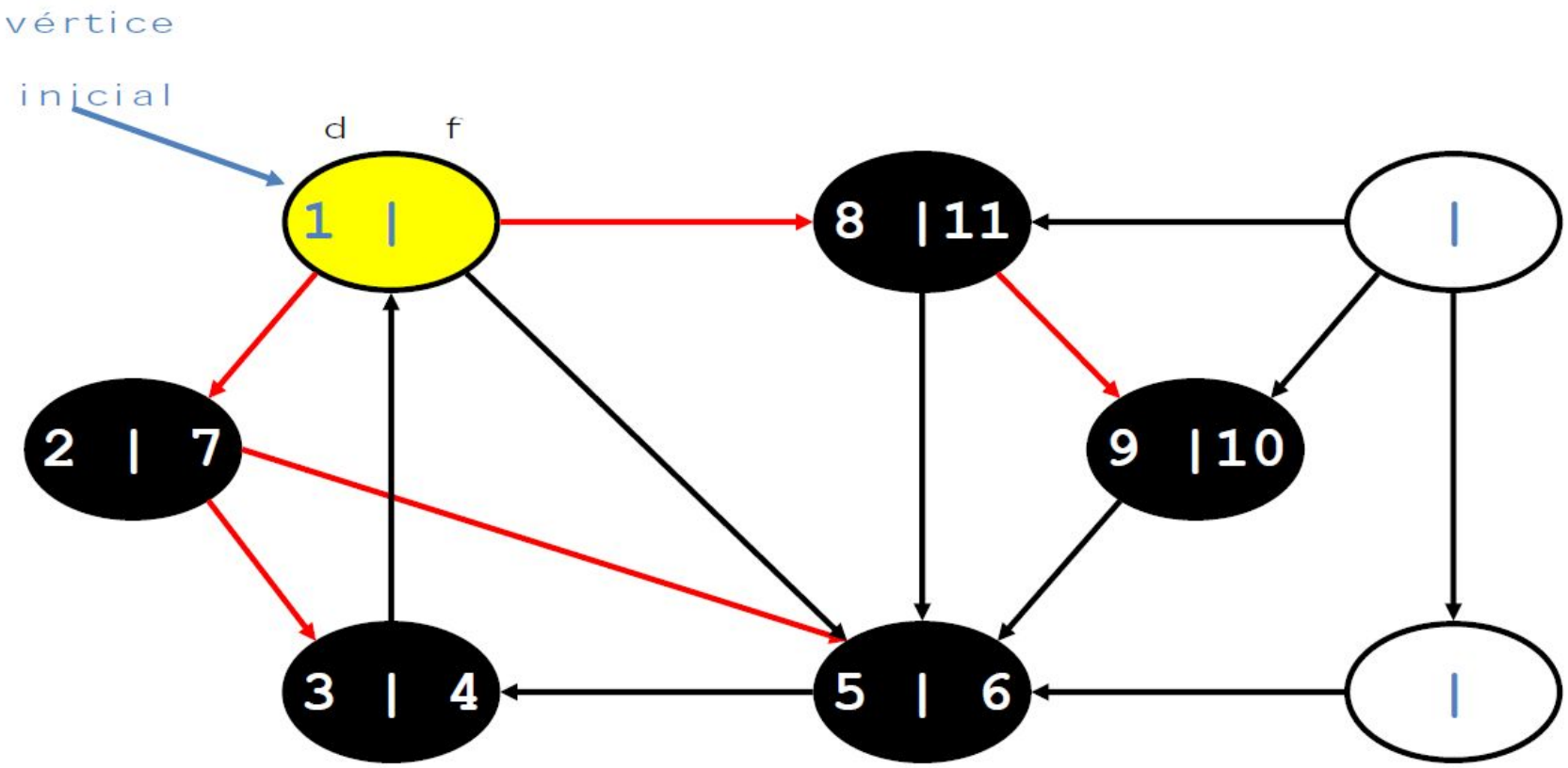

```
DFS_Visit(u)
{
    u->color = AMARILLO;
    tiempo = tiempo+1;
    u->d = tiempo;
    for each v ∈ u->Adj[]
    {
        if (v->color== BLANCO)
            DFS_Visit(v);
    }
    u->color = NEGRO;
    tiempo = tiempo +1;
    u->f = tiempo;
}
```

Traza de Recorrido en Profundidad DFS (11/17)



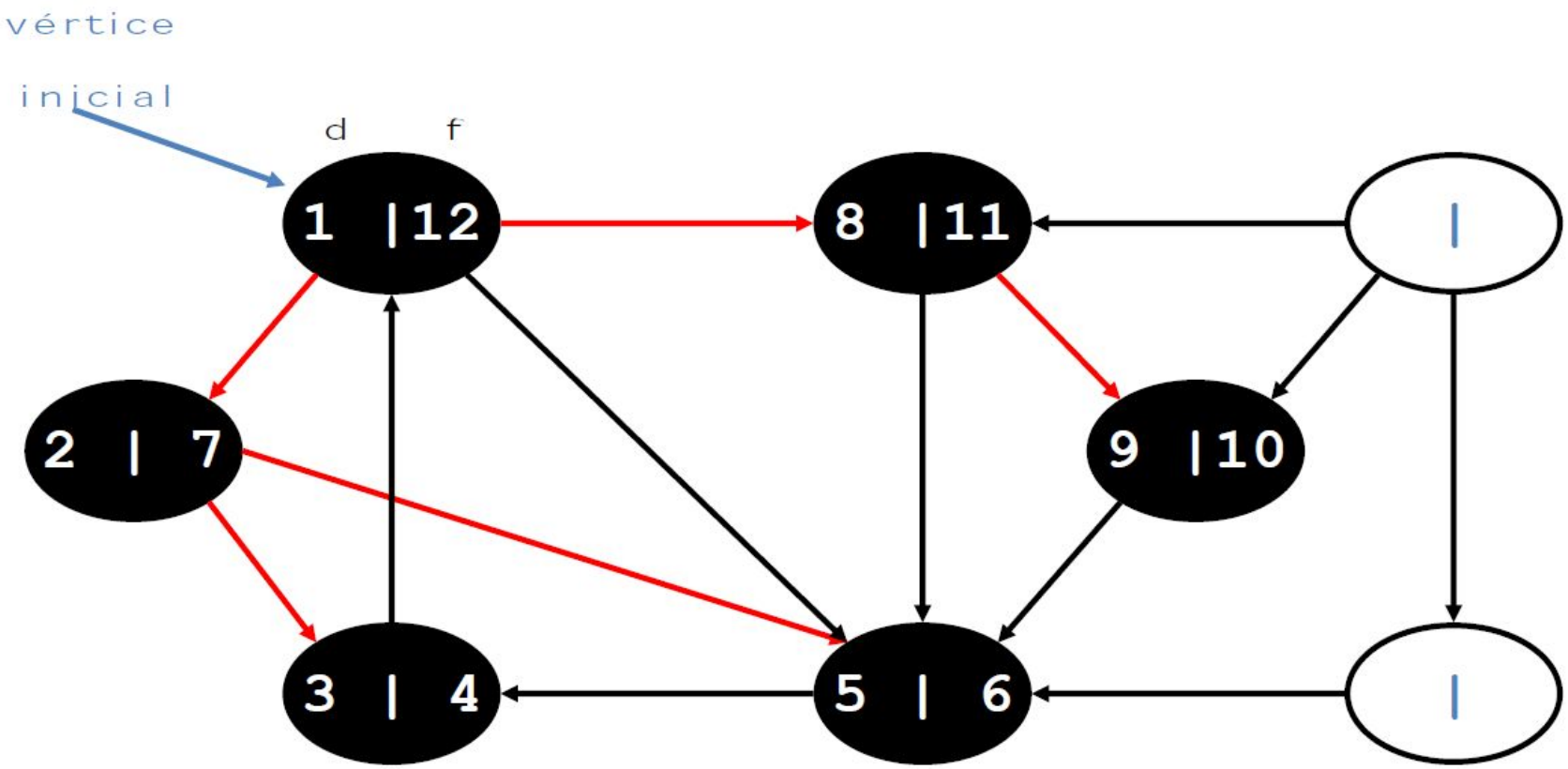

```
DFS_Visit(u)
{
    u->color = AMARILLO;
    tiempo = tiempo+1;
    u->d = tiempo;
    for each v ∈ u->Adj[]
    {
        if (v->color== BLANCO)
            DFS_Visit(v);
    }
    u->color = NEGRO;
    tiempo = tiempo +1;
    u->f = tiempo;
}
```

Traza de Recorrido en Profundidad DFS (12/17)



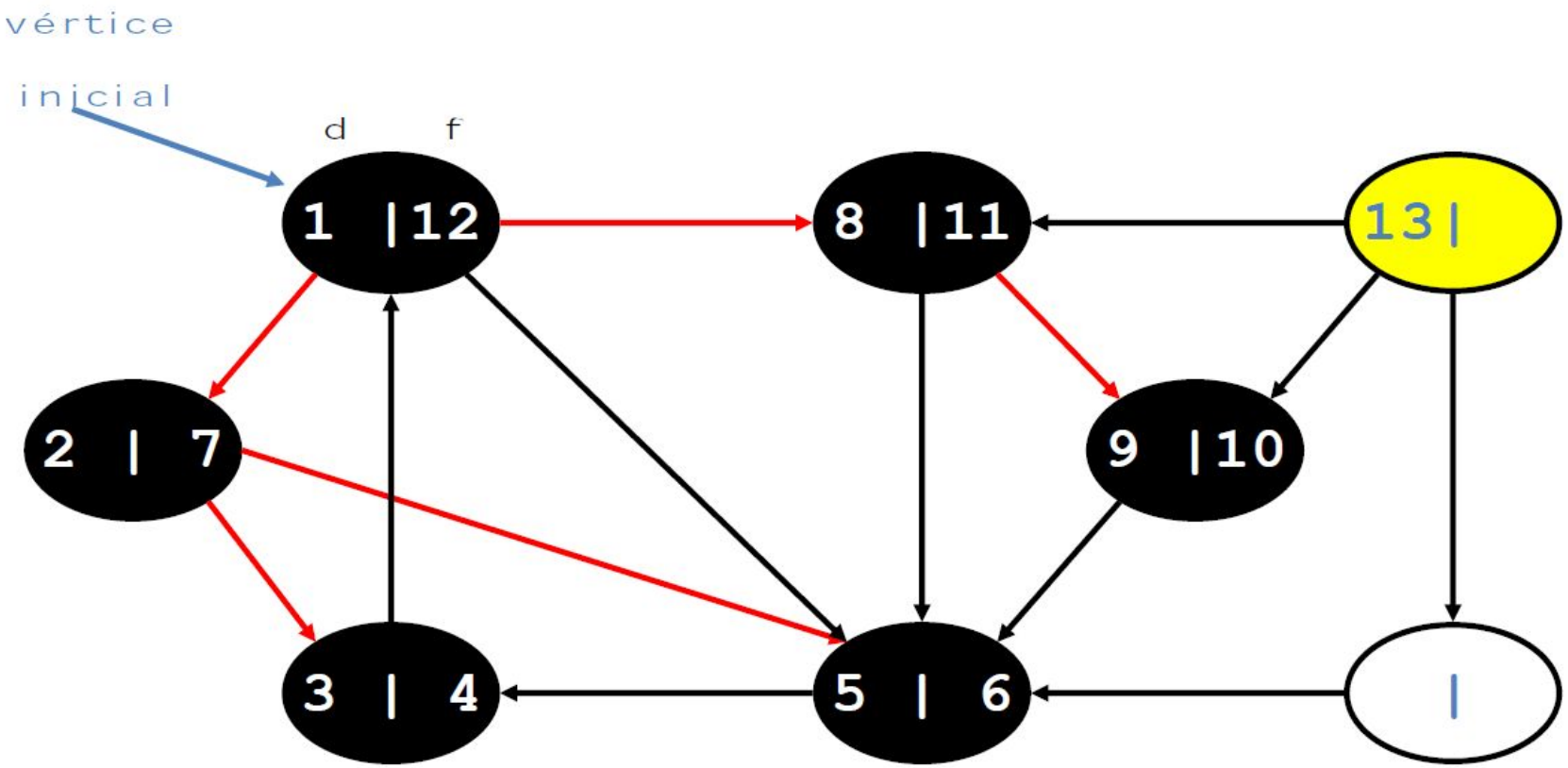

```
DFS_Visit(u)
{
    u->color = AMARILLO;
    tiempo = tiempo+1;
    u->d = tiempo;
    for each v ∈ u->Adj[]
    {
        if (v->color== BLANCO)
            DFS_Visit(v);
    }
    u->color = NEGRO;
    tiempo = tiempo +1;
    u->f = tiempo;
}
```

Traza de Recorrido en Profundidad DFS (13/17)




```
DFS_Visit(u)
{
    u->color = AMARILLO;
    tiempo = tiempo+1;
    u->d = tiempo;
    for each v ∈ u->Adj[]
    {
        if (v->color== BLANCO)
            DFS_Visit(v);
    }
    u->color = NEGRO;
    tiempo = tiempo +1;
    u->f = tiempo;
}
```

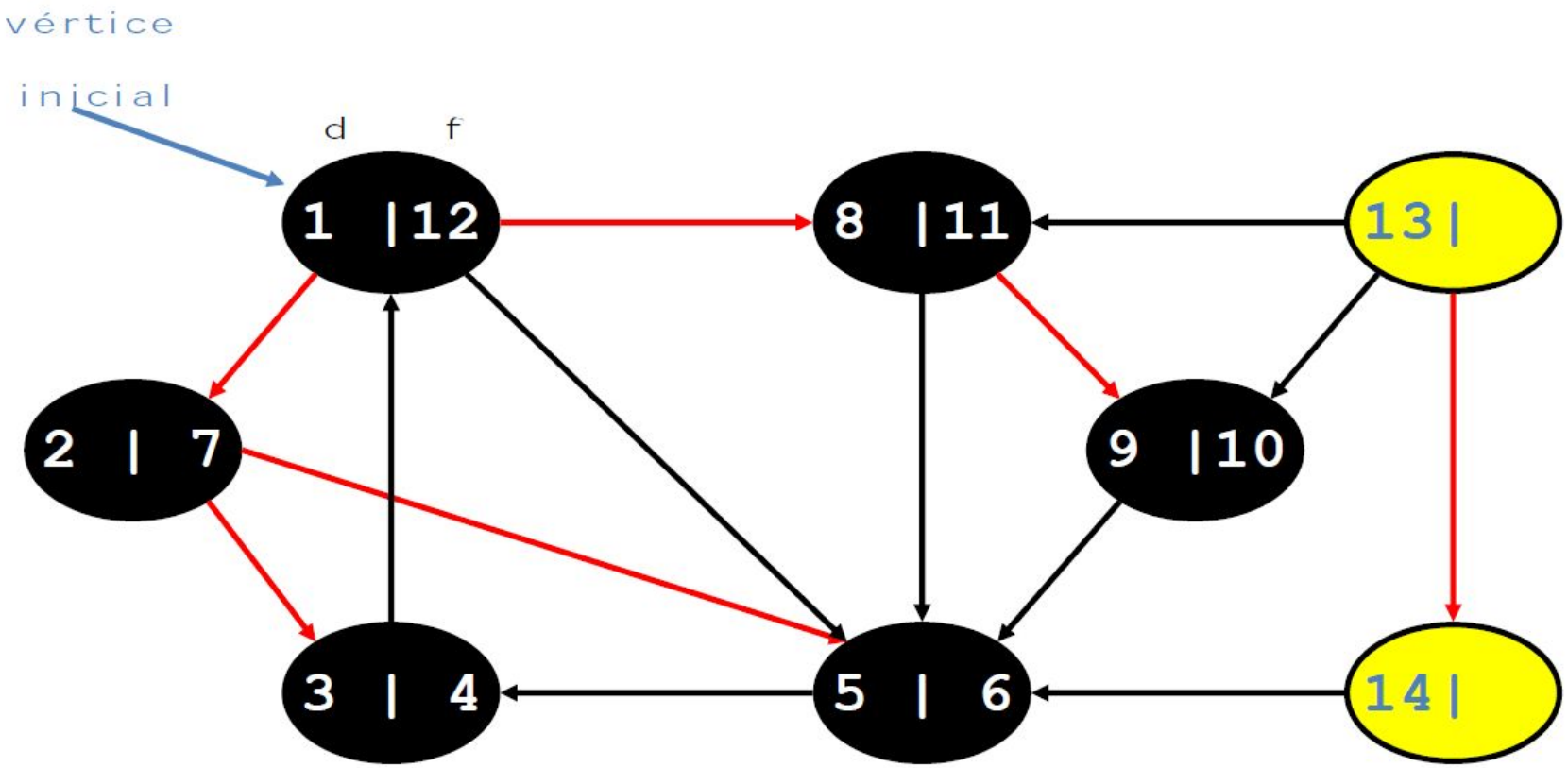
Traza de Recorrido en Profundidad DFS (14/17)




```
DFS(G)
{
    for each vértice u ∈ G->V
    {
        u->color = BLANCO;
    }
    tiempo= 0;
    for each vértice u ∈ G->V
    {
      if (u->color == BLANCO)
          DFS_Visit(u);
    }
}
```

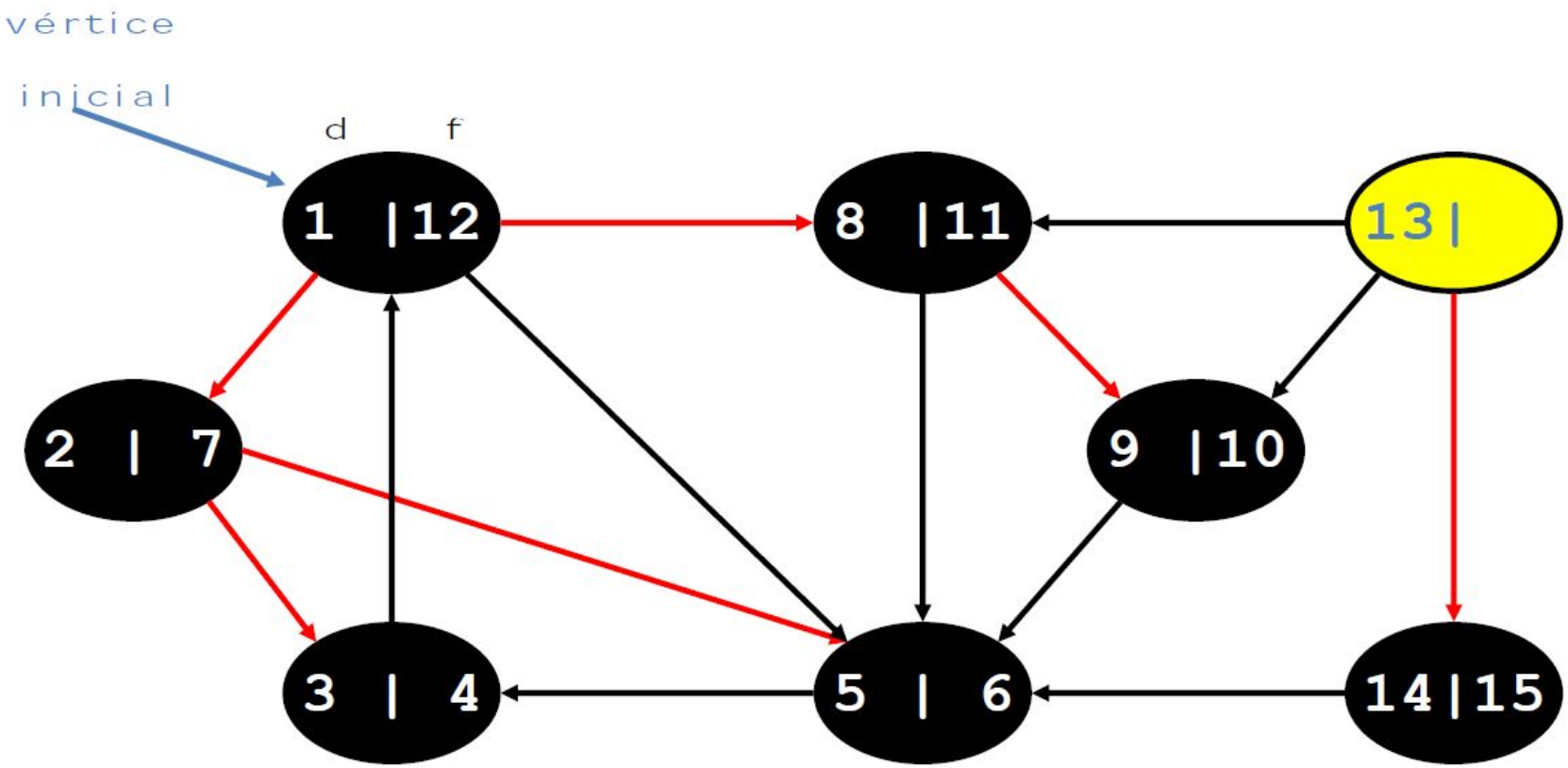
```
DFS_Visit(u)
{
    u->color = AMARILLO;
    tiempo = tiempo+1;
    u->d = tiempo;
    for each v ∈ u->Adj[]
    {
        if (v->color== BLANCO)
            DFS_Visit(v);
    }
    u->color = NEGRO;
    tiempo = tiempo +1;
    u->f = tiempo;
}
```

Traza de Recorrido en Profundidad DFS (15/17)



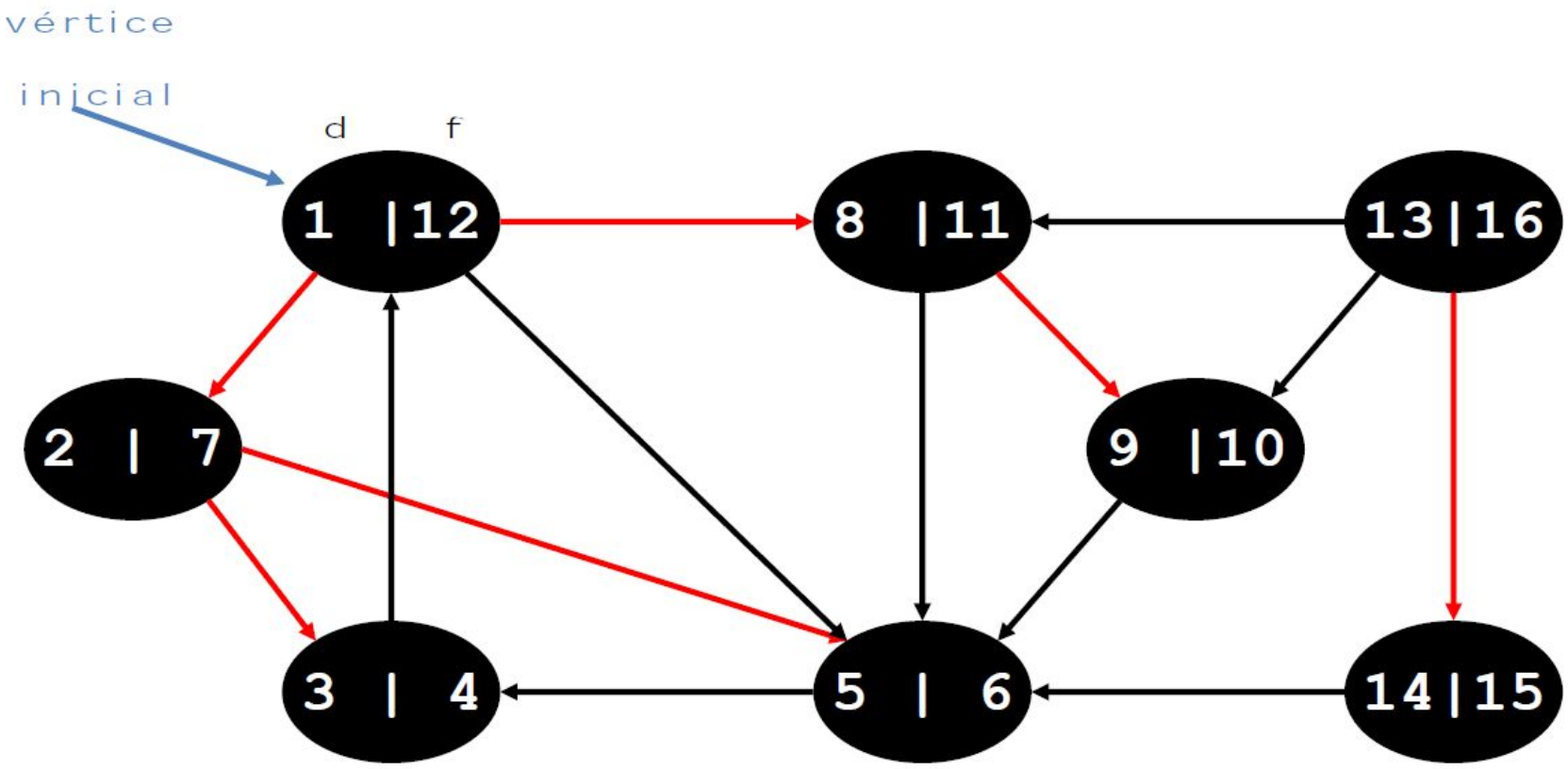

```
DFS_Visit(u)
{
    u->color = AMARILLO;
    tiempo = tiempo+1;
    u->d = tiempo;
    for each v ∈ u->Adj[]
    {
        if (v->color== BLANCO)
            DFS_Visit(v);
    }
    u->color = NEGRO;
    tiempo = tiempo +1;
    u->f = tiempo;
}
```

Traza de Recorrido en Profundidad DFS (16/17)




```
DFS_Visit(u)
{
    u->color = AMARILLO;
    tiempo = tiempo+1;
    u->d = tiempo;
    for each v ∈ u->Adj[]
    {
        if (v->color== BLANCO)
            DFS_Visit(v);
    }
    u->color = NEGRO;
    tiempo = tiempo +1;
    u->f = tiempo;
}
```

Traza de Recorrido en Profundidad DFS (17/17)



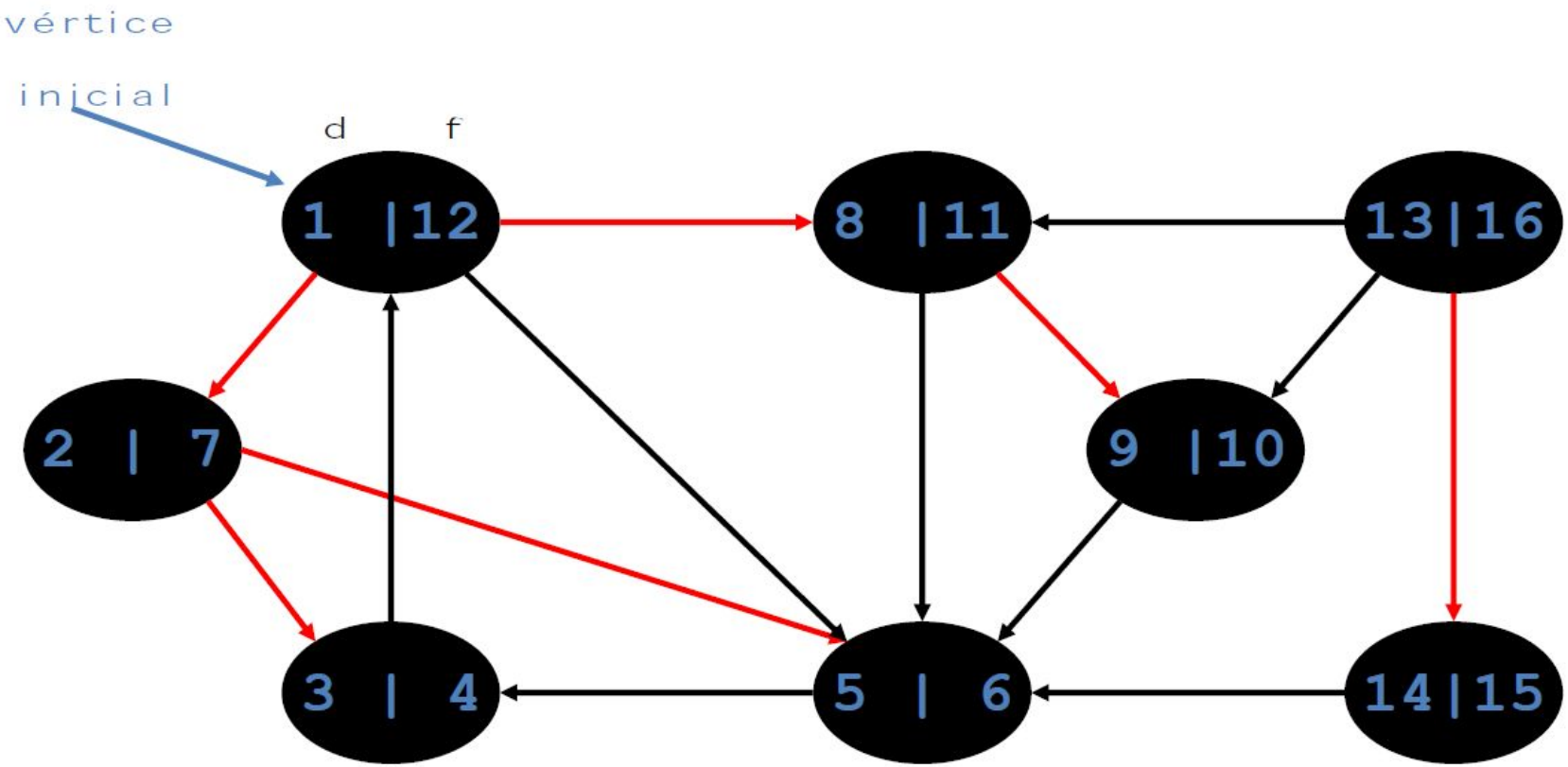

```
DFS(G)
{
   for each vértice u ∈ G->V
   {
      u->color = BLANCO;
   }
   tiempo= 0;
   for each vértice u ∈ G->V
   {
     if (u->color == BLANCO)
         DFS_Visit(u);
   }
}

DFS_Visit(u)
{
    u->color = AMARILLO;
    tiempo = tiempo+1;
    u->d = tiempo;
    for each v ∈ u->Adj[]
    {
       if (v->color== BLANCO)
          DFS_Visit(v);
    }
    u->color = NEGRO;
    tiempo = tiempo +1;
    u->f = tiempo;
}
```

DFS: Clases de Arcos

DFS introduce una distinción importante entre arcos en el grafo original:

- *Arcos Tree* : encuentra vértices nuevos (blancos)
  - Los arcos tree forman un bosque abarcador

Clasificación de Arcos: TREE



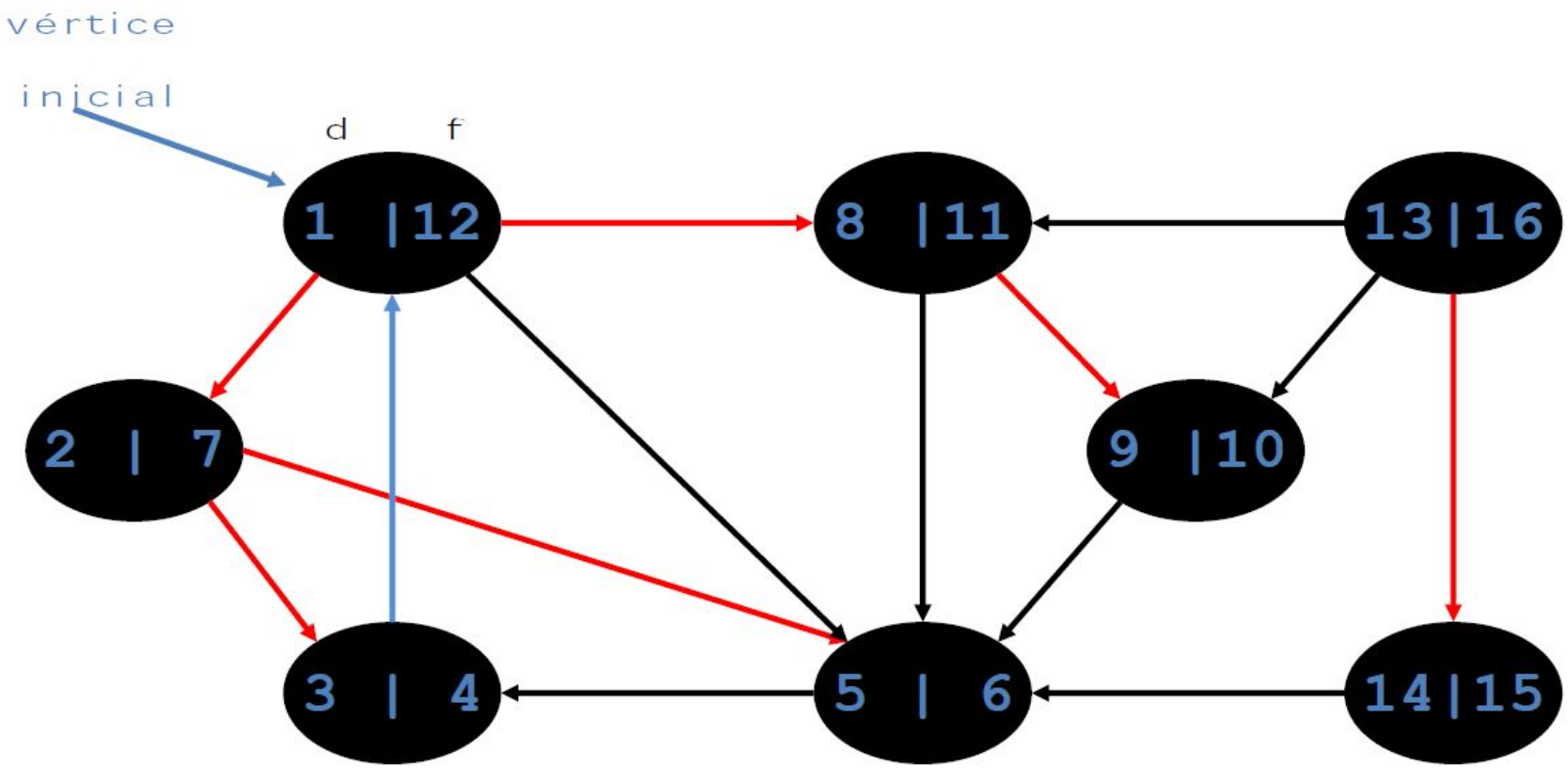
Arco Tree

DFS: Clases de arcos

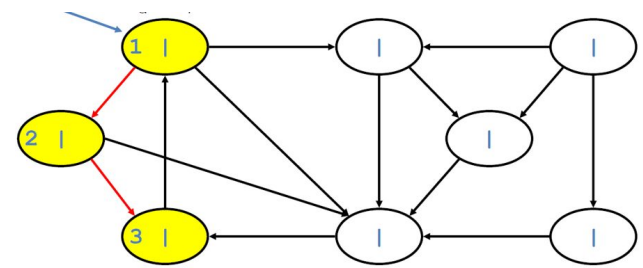
DFS introduce una distinción importante entre arcos en el grafo original:

- *Arcos Back*: desde el descendiente al ancestro
  - Encuentra un vértice amarillo (de amarillo a amarillo)

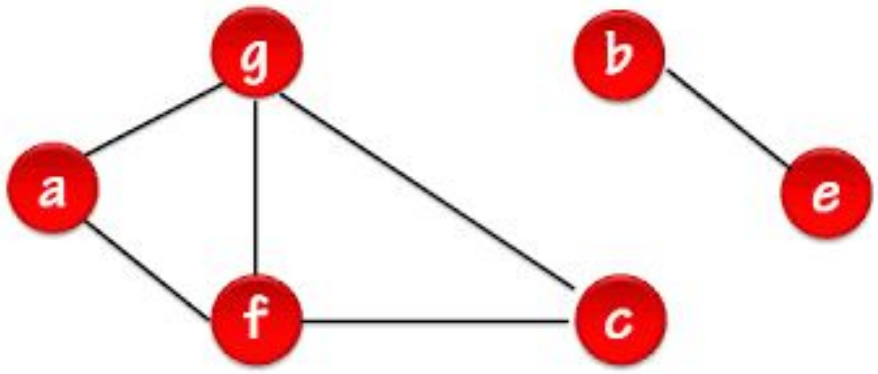
Clasificación de Arcos: Back



Arco Tree Arco Back

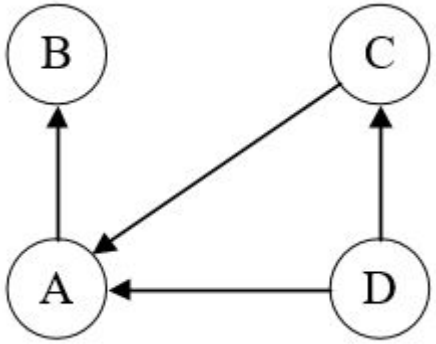


Se visitan todos los nodos sin importar la forma del grafo.




```
DFS(G)
{
    for each vértice u ∈ G->V
    {
        u->color = BLANCO;
    }
    tiempo= 0;
    for each vértice u ∈ G->V
    {
      if (u->color == BLANCO)
          DFS_Visit(u);
    }
}
```

```
DFS_Visit(u)
{
    u->color = AMARILLO;
    tiempo = tiempo+1;
    u->d = tiempo;
    for each v ∈ u->Adj[]
    {
        if (v->color== BLANCO)
            DFS_Visit(v);
    }
    u->color = NEGRO;
    tiempo = tiempo +1;
    u->f = tiempo;
}
```



DFS y Ciclos en los grafos

Un Grafo es acíclico si y sólo si en el recorrido DFS no se encuentran arcos de tipo "back". O sea nunca se intenta ir de un vértice amarillo a otro amarillo.

Podemos usar el recorrido DFS entonces para hacer lo que se conoce como test de aciclicidad de un grafo, que es determinar si el grafo es cíclico o acíclico.

Cómo modificarías el pseudocódigo para detectar ciclos?

```
DFS(G)
{
    for each vértice u ∈ G->V
    {
        u->color = BLANCO;
    }
    tiempo= 0;
    for each vértice u ∈ G->V
    {
        if (u->color== BLANCO)
            DFS_Visit(u);
    }
}
```

```
DFS_Visit(u)
{
    u->color = AMARILLO;
    tiempo= tiempo+1;
    u->d = tiempo;
    for each v ∈ u->Adj[]
    {
        if (v->color== BLANCO)
            DFS_Visit(v);
    }
    u->color = NEGRO;
    tiempo = tiempo+1;
    u->f = tiempo;
}
```

```
else
  if (v->color == AMARILLO)
     print("Hay Ciclo!");
```

DFS: Complejidad Computacional

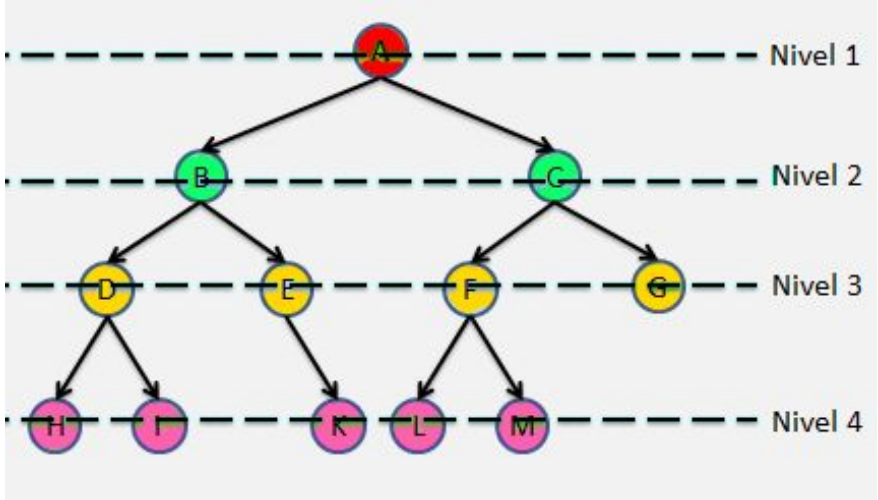
Tiempo de ejecución?

$O(|V| + |A|)$

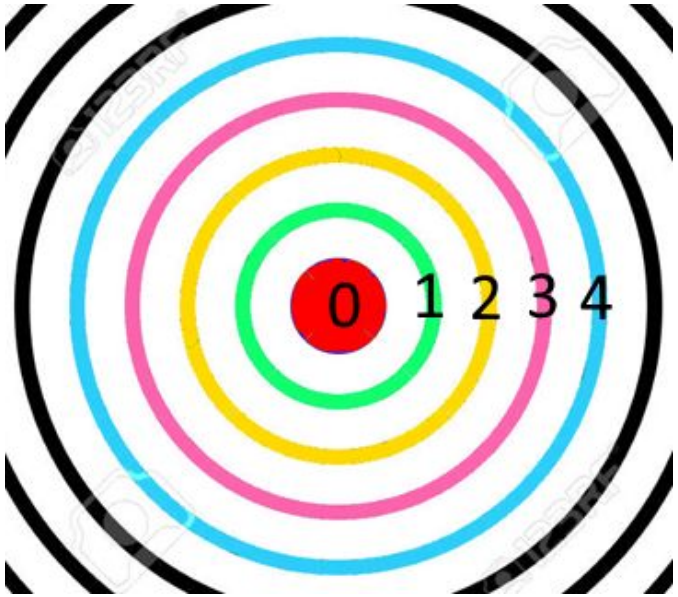
Con Listas de Adyacencia

BFS (Breadth-First Search)
Recorrido en Amplitud

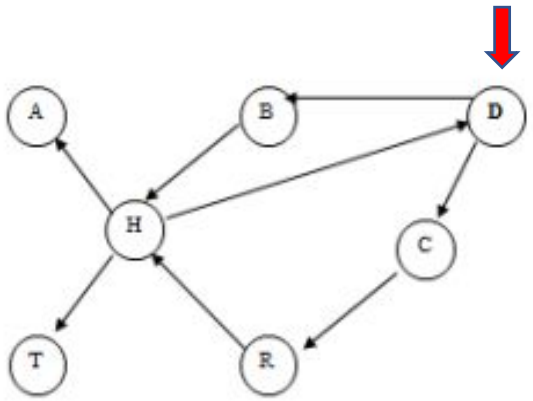
- Es una generalización del recorrido por niveles de un árbol.

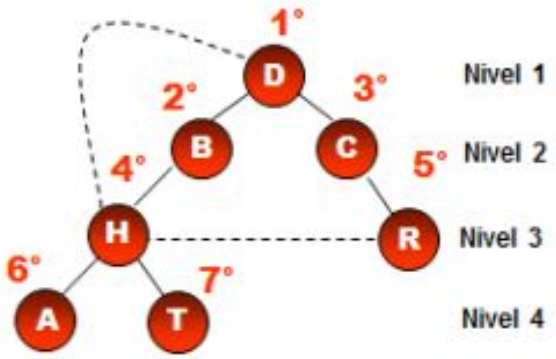


- Se realiza un recorrido por capas del grafo.



BFS





Recorrido desde Vertice por anchura desde vertice D ={D, B, C, H, R, A, T }

Para hacerlo nos valemos de una estructura de fila.

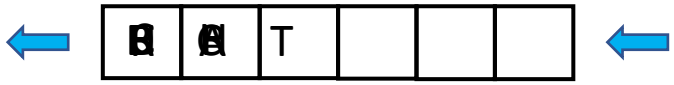
BFS (**G**):
  Vaciar la fila **F**.
  Para cada vértices v de **G**
    Marcar **v** como NO_VISITADO.

  Para cada vértice v de **G**
    Si **v** es NO_VISITADO:
      BFS_visit(**G**,**v**)

BFS_visit(**G, s**):
  Marcar el vértice **s** como VISITADO.
  Agregar **s** a la fila **F**.
  Mientras la fila **F** no esté vacía
    Tomamos vértice **x** de la fila,
    Para cada vértice **y** adyacente a **x**:
      Si **y** es NO_VISITADO :
        Marcar el vértice **y** como VISITADO.
        Agregar **y** a la fila F.

1) D
2) BC
3) CH
4) HR
5) RAT
6) AT
7) T
8)

Es un planteo iterativo, no recursivo.

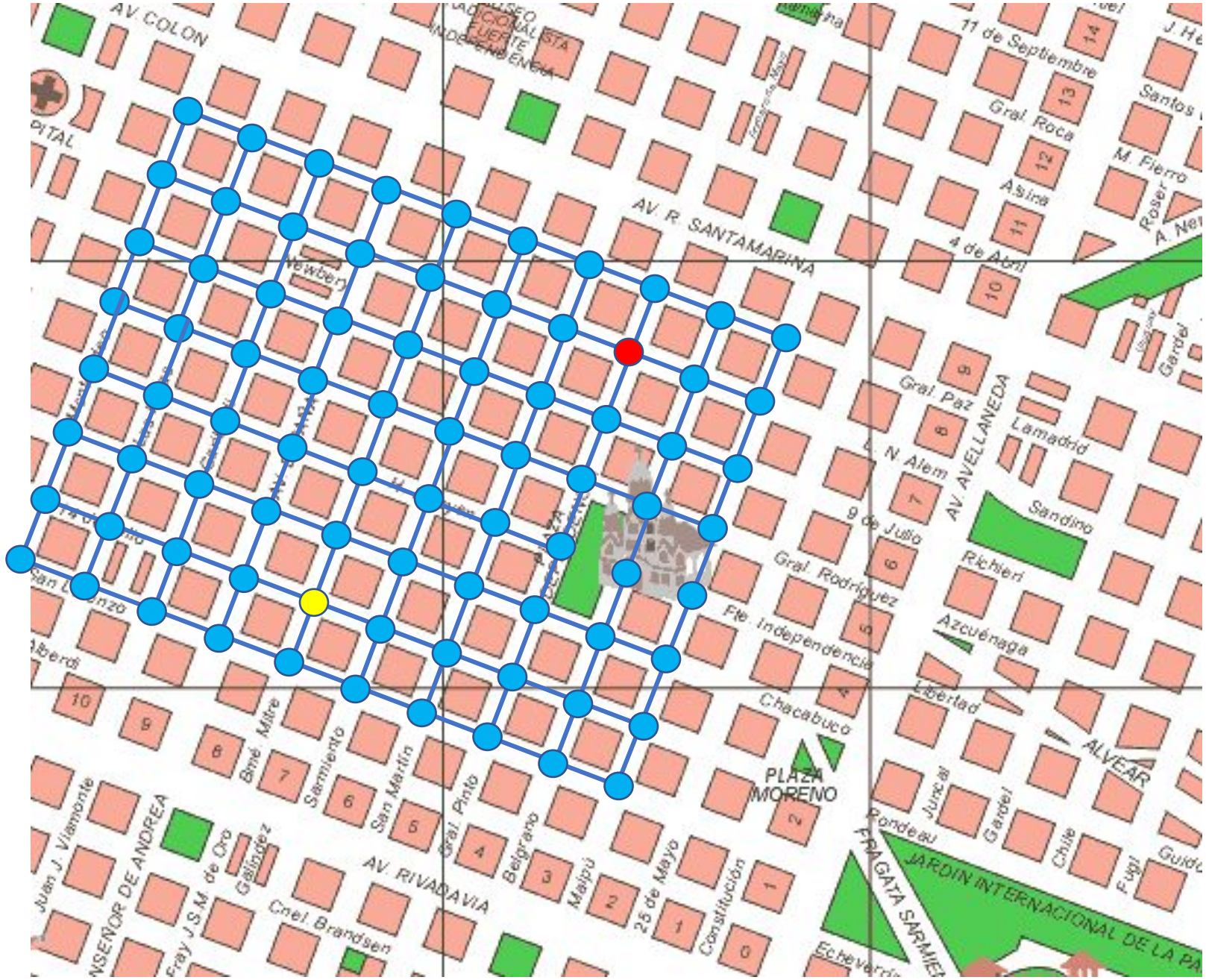


Problema clásico

- Cómo encontrar el camino con menor longitud (menor cantidad de arcos) entre un vértice v y otro vértice w.

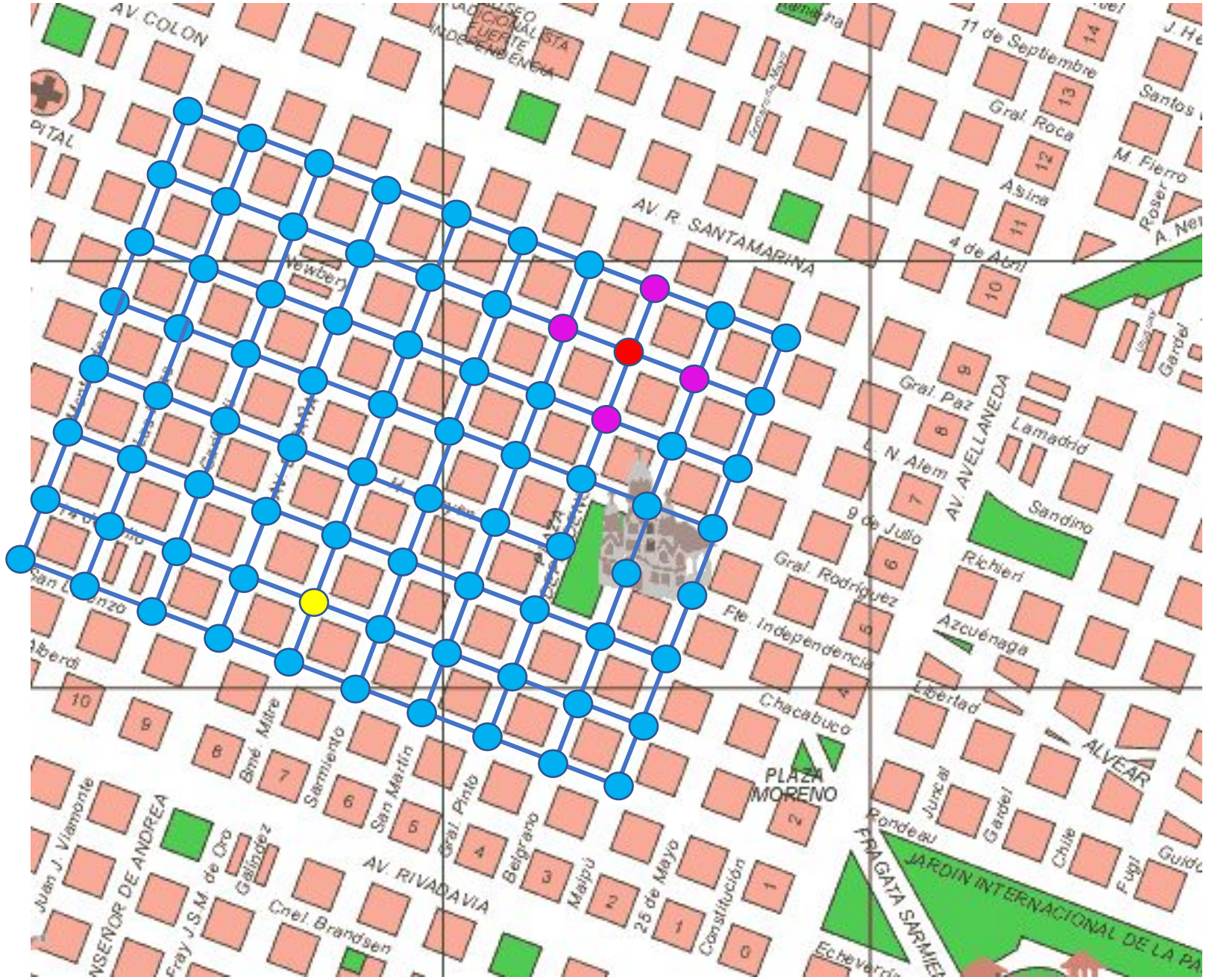
- Estrategia: Visitar primero todos los de distancia 1 desde v, luego los de distancia 2, luego los de 3, y asi sucesivamente. Verificar Si se encuentra w, y en qué nivel.
- Para esto hacemos un BFS desde v controlando cuando llegamos a w.

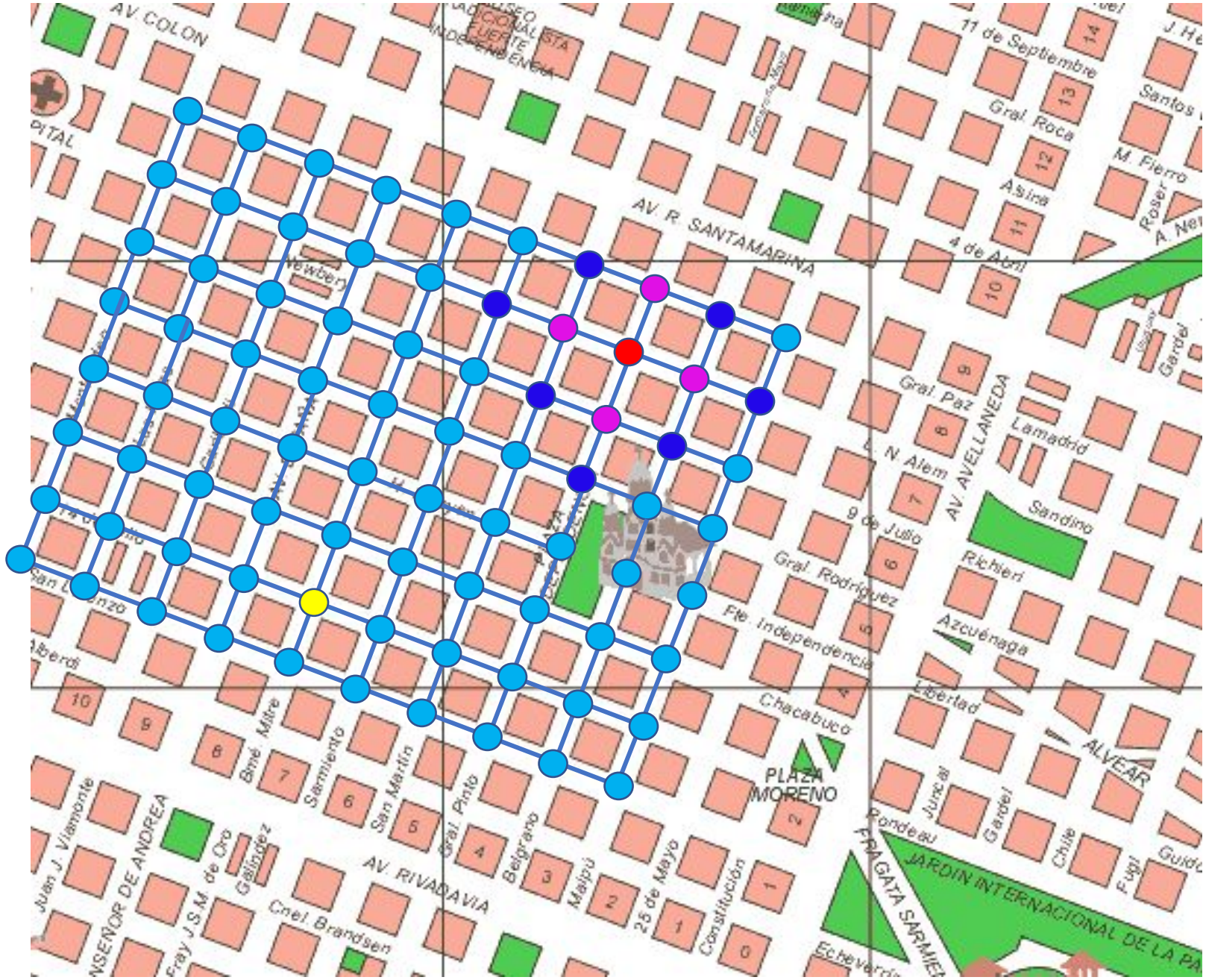
AV. COLON
11 de Septiembre
Gral. Roca
Asirá
M. Fierro
AV. R. SANTAMARINA
4 de Abril
Newbery
Montevideo
Las Heras
Garibaldi
AV. ESPAÑA
H. Irigoyen
PLAZA INDEPENDENCIA
Gral. Paz
N. Alem
AV. AVELLANEDA
Lamadrid
Sandino
9 de Julio
14 de Julio
Richieri
Gral. Rodríguez
San Lorenzo
Azcuénaga
Fte. Independencia
Chacabuco
Libertad
ALVEAR
Bmé. Mitre
Sarmiento
San Martín
Gral. Pinto
PLAZA MORENO
Juncal
Gardel
Belgrano
Malpú
25 de Mayo
Constitución
Pondeau
Chile
Fugi
FRAGATA SARMIENTO
JARDIN INTERNACIONAL DE LA PA
AV. RIVADAVIA
Cnel. Brandsen
Juan J. Viamonte
MONSEÑOR DE ANDREA
Fray J.S.M. de Oro
Galindez
Echeverría

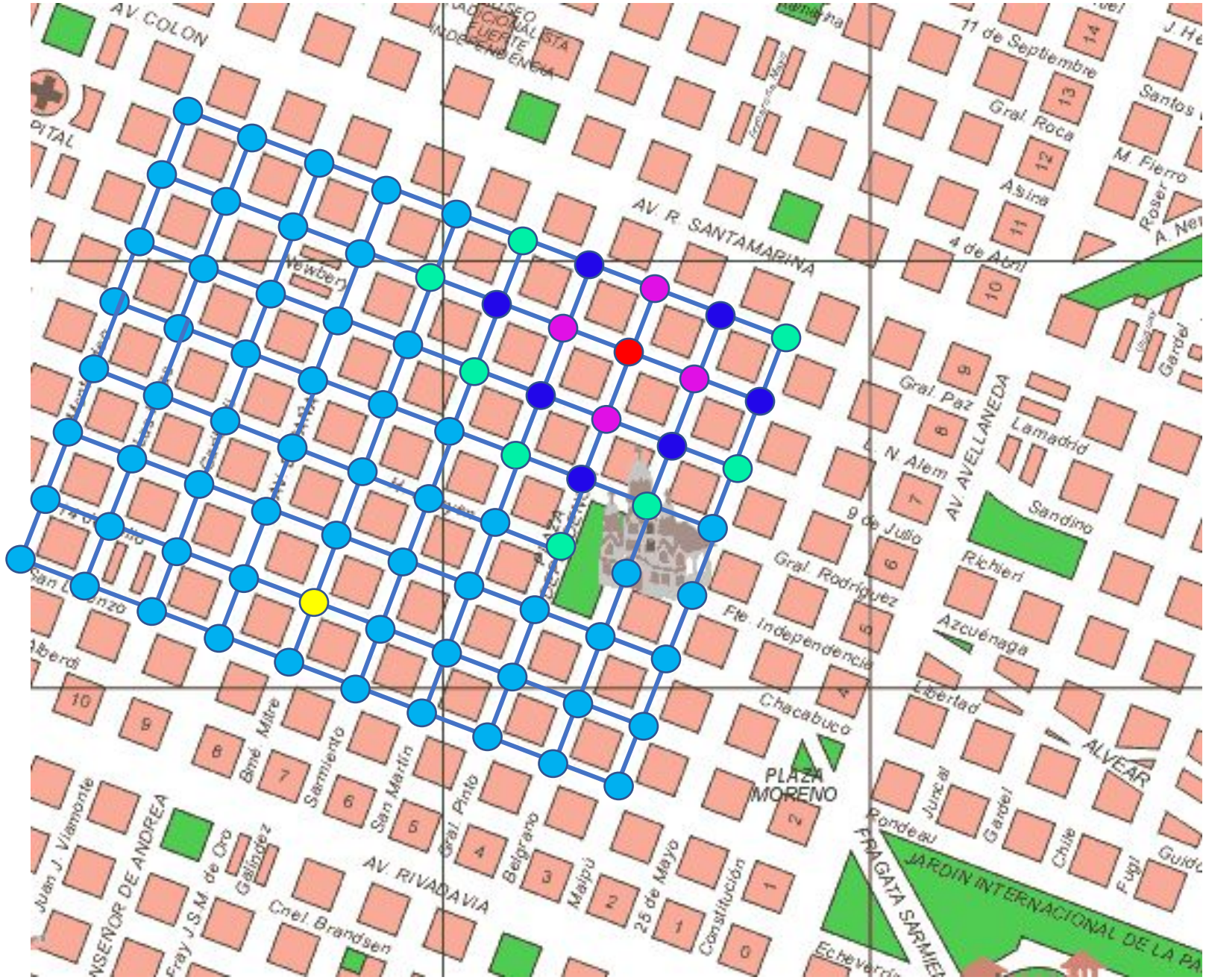


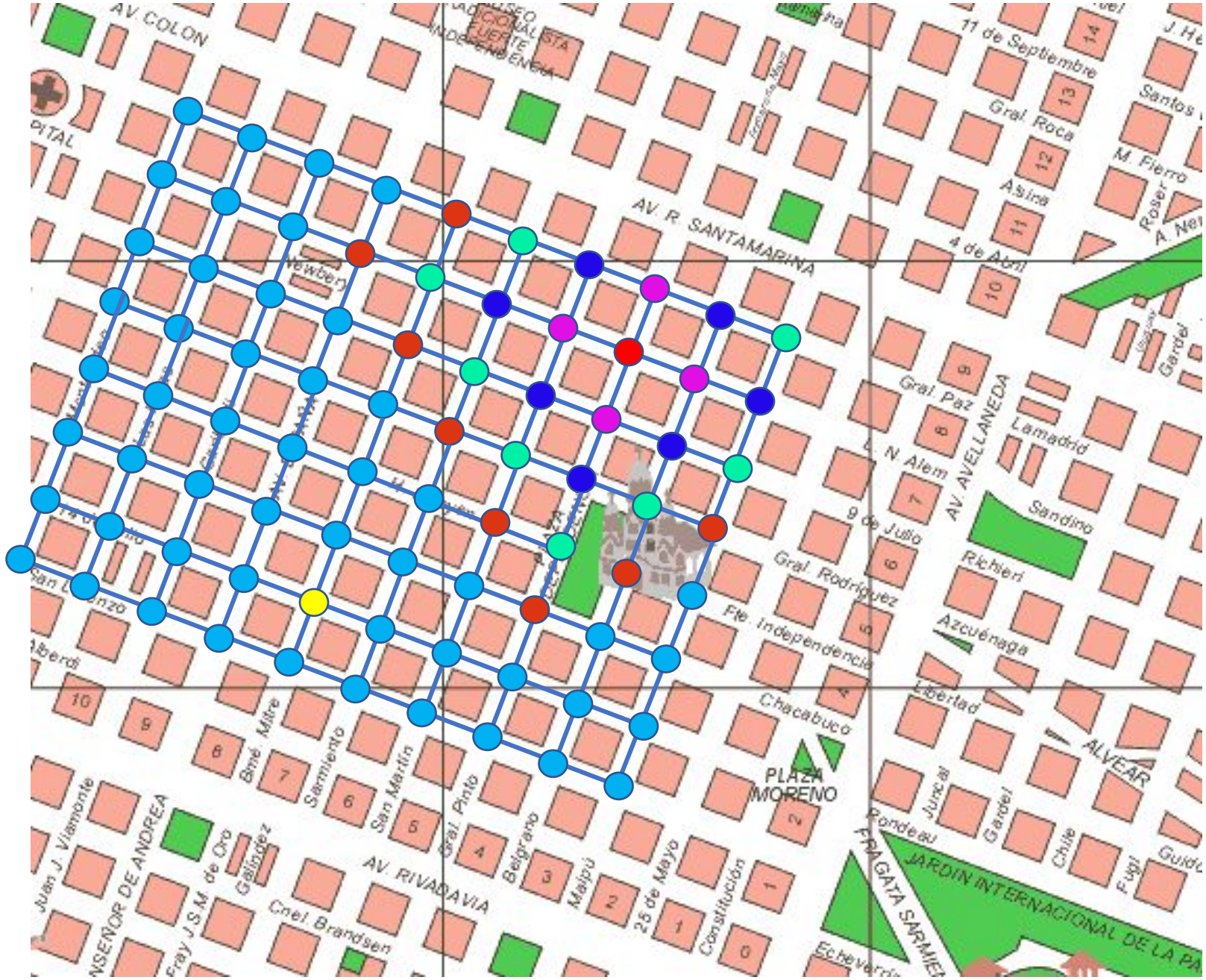
Cuál es la menor cantidad de cuadras a caminar para llegar desde rojo hasta amarillo?

Aplicamos BFS desde rojo hasta encontrar el amarillo.


AV. COLON
11 de Septiembre
Gral. Roca
Asirá
4 de Abril
AV. R. SANTAMARINA
Newbery
Gral. Paz
AV. AVELLANEDA
Lamadrid
N. Alem
9 de Julio
Sandino
Richieri
Gral. Rodríguez
Fte. Independencia
Azcuénaga
Libertad
Chacabuco
PLAZA MORENO
ALVEAR
Juncal
Gardel
Chile
Fugi
Pondeau
FRAGATA SARMIENTO
JARDIN INTERNACIONAL DE LA PA
Echeverría
Belgrano
Malpú
25 de Mayo
Constitución
Gral. Pinto
San Martín
Sarmiento
Bmé. Mitre
AV. RIVADAVIA
Cnel. Brandsen
Juan J. Viamonte
MONSEÑOR DE ANDREA
Fray J.S.M. de Oro
Galindez
Aberdi


AV. COLON
11 de Septiembre
Gral. Roca
Asirá
4 de Abril
AV. R. SANTAMARINA
M. Fierro
Santos
Newbery
Gral. Paz
AV. AVELLANEDA
Lamadrid
N. Alem
9 de Julio
Sandino
Richieri
Gral. Rodríguez
Azcuénaga
Fte. Independencia
Chacabuco
Libertad
ALVEAR
PLAZA MORENO
Bmé. Mitre
Sarmiento
San Martín
Gral. Pinto
Belgrano
Malpú
25 de Mayo
Constitución
AV. RIVADAVIA
Cnel. Brandsen
Juan J. Viamonte
MONSEÑOR DE ANDREA
Fray J.S.M. de Oro
Galindez
Rondeau
Junín
Gardel
Chile
Fugi
FRAGATA SARMIENTO
JARDIN INTERNACIONAL DE LA PA
Echeverría


AV. COLON
11 de Septiembre
Gral. Roca
Asirá
4 de Abril
AV. R. SANTAMARINA
Gral. Paz
AV. AVELLANEDA
Lamadrid
N. Alem
9 de Julio
Sandino
Richieri
Gral. Rodríguez
Azcuénaga
Fte. Independencia
Chacabuco
Libertad
ALVEAR
PLAZA MORENO
Rondeau
Junín
Gardel
Chile
Fugi
JARDIN INTERNACIONAL DE LA PA
FRAGATA SARMIE
Echeverri
Bmé. Mitre
Sarmiento
San Martín
Gral. Pinto
Belgrano
Malpú
25 de Mayo
Constitución
AV. RIVADAVIA
Cnel. Brandsen
Juan J. Viamonte
NSEÑOR DE ANDREA
Gray J.S.M. de Oro
Galindez
Newbery
Roser
M. Fierro
Santos
Garde



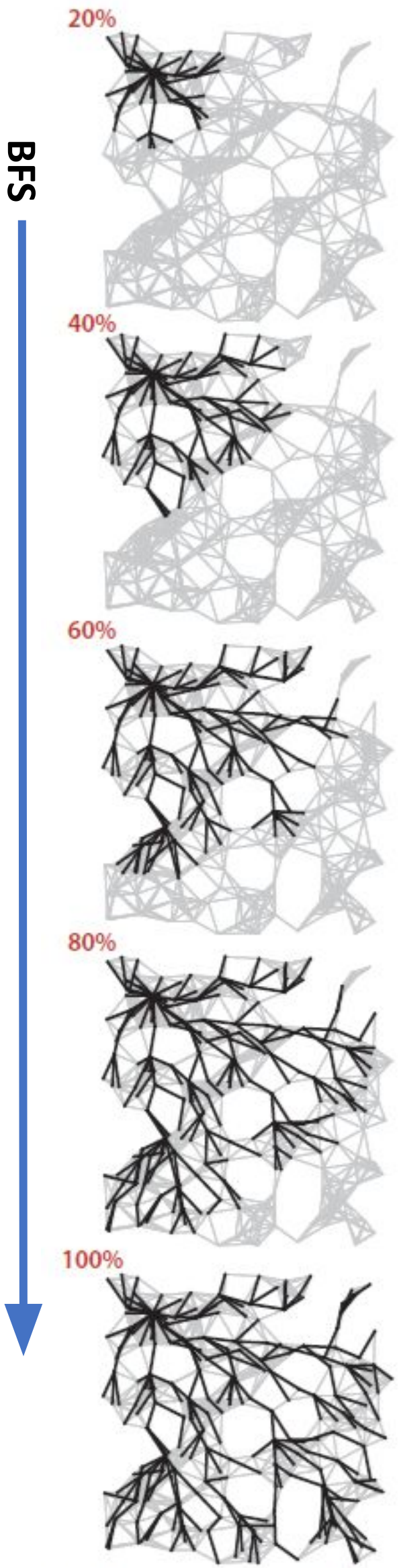
Encontramos al amarillo en el nivel 8 (distancia 8 desde el orígen).
O sea caminaremos como mínimo 8 cuadras.

BFS: Complejidad Computacional

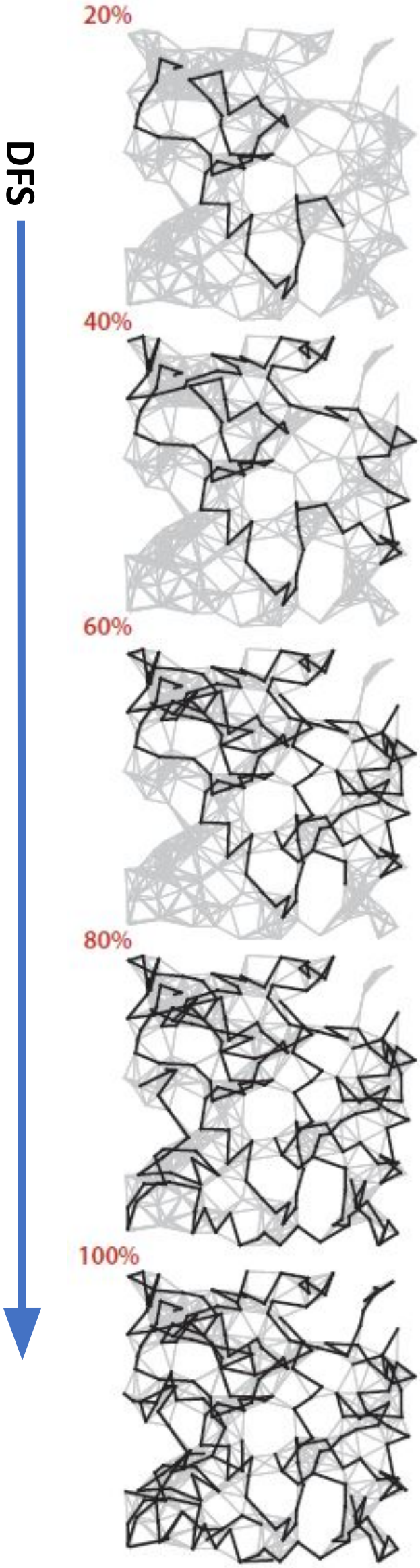
Tiempo de ejecución?

$O(|V| + |A|)$

Con Listas de Adyacencia



BFS for shortest paths (250 vertices)



DFS for paths (250 vertices)